

ROCQ BOTTOM: Beyond Soundness with Precise Reasoning about Analysis Precision^{*}

Matthieu Lemerre

CEA, LIST

Abstract. Imprecision in abstract transfer functions has concrete costs: verifiers fail to discharge assertions that hold, and compilers miss optimisations they could legitimately apply. Yet soundness-only certification cannot distinguish a precise transfer function from the constant- $\top$ one. We show that a principled, precision-aware design, rooted in standard abstract-interpretation theory, is achievable in plain constructive logic. We present ROCQ BOTTOM, a Rocq library that hierarchises several “best-precision” notions (γ -exactness, best abstraction, α -completeness), and provides an algebraic hierarchy of Galois connections, abstract lattices, and combinators that compose precision properties from components. The framework encodes Galois connections via α -relations rather than total α functions, side-stepping the constructivity issue that has historically led mechanised verifications of executable abstract interpreters to avoid Galois connections.

We instantiate the framework on quadrivalent booleans, known bits, Granger congruences, possibly-unbounded integer intervals, and a reduced product of intervals and congruences, verifying optimality of many transfer functions for comparisons, arithmetic, or bitwise operations. Exactness of individual operations holds in many cases, yet we also prove that it can be lost by combination: the best addition on the reduced product is not exact although both component additions are. Finally, optimality proofs drive the implementation toward maximally-precise yet efficient code.

1 Introduction

Abstract interpretation is built on a soundness contract: a transfer function must overapproximate the concrete semantics. But soundness alone is a weak guarantee — the constant- $\top$ transfer function satisfies it and is useless. In practice, the quality of an analysis depends on how *precise* its transfer functions are.

The gap is not academic. We were led to this work when implementing a bitvector abstraction library on top of a library of integer abstractions, and assumed that some interval wrapping was unnecessary based on optimality assumptions that did not hold in that library. Without optimality guarantees, a consumer of the library cannot reason about the precision of the analysis, nor optimise its code around important invariants. Imprecision is also a common

^{*} Accepted at the 32nd Static Analysis Symposium (SAS 2026). This is the author’s version of the work, posted before publication.

source of hard-to-find bugs [1] that silently disable compiler optimisations or prevent the automated verification of programs one would expect to prove.

Abstract interpretation provides a *principled* framework for reasoning about best precision. The canonical setting is the notion of *best transformer* defined via Galois connections by Cousot and Cousot [2], which specifies what a transfer function should ideally compute. It does not, however, specify how to write such a transfer function. There are some abstractions where the best abstract transformer can be computed automatically using symbolic methods [3–5], and some approaches that use guided random synthesis to propose transfer functions that are *a posteriori* checked for precision [6–8]. These approaches are useful but require the problem to be expressed in a decidable logic (e.g., bitvectors), which prevents reasoning about domains with infinite ascending chains, such as integer intervals or congruences. In the latter case, the produced algorithm can work but is neither readable nor efficient, since synthesis decouples the resulting code from any explanation of its precision; furthermore only precision according to the concrete order is considered, which, as we show, is a weak notion.

Therefore, there is a need for a *principled approach that can prove optimality of arbitrary transfer functions and drive implementations toward maximally-precise yet efficient code*, rooted in abstract interpretation theory.

To deliver such guarantees, we turn to interactive verification in a proof assistant; specifically, Rocq. Considerable prior work has formalised parts or whole abstract interpreters in proof assistants, but invariably along one of three lines: it relies on additional, non-constructive axioms [9]; it adopts an alternative constructive Galois-connection theory [10] that cannot represent domains that *aggregate* information about a set, like intervals or congruences; or it adopts a γ -only formalisation [11], which can verify soundness but says nothing about precision [12–16].

There are good reasons for the popularity of the latter soundness-only style. First, soundness is arguably more important than precision: if an analysis is unsound, its precision does not matter. Second, soundness proofs are simpler than best-precision proofs, and formal verification is already a large effort. We argue that this compromise should be reconsidered: imprecision leads to silent misoptimisations and broken invariants that are hard to debug, while interactive verification has matured considerably.

However, there are also technical obstacles that have prevented mechanized verifications based on Galois connections. First, the total $\alpha : \wp(C) \rightarrow A$ does not always exist: there is no best polyhedron approximating a circle, and no best rational interval approximating $\{\pi\}$. Even when a best abstract element exists, Rocq’s separation between the computable and the logical world prevents defining α , which extracts a computable object (the abstract element) from a purely-logical one (the set, described as a predicate). We propose a simple fix that, surprisingly, has not been explored before: define α -relations instead of α -functions. As we show, a relation $a \sim S$ between an abstract element and a concrete set suffices for every precision proof we need, while the rewriting machinery one expects from a Galois connection survives unchanged. We further

identify a hierarchy of precision relations (`OVERAPPROXIMATES`, `MOSTPRECISE`, `BESTABSTRACTION/ISALPHA`, `EXACTLYREPRESENTS`; see §3), and prove the strongest one each transfer function admits. *Exactness*, often thought to be too strong and thus less interesting [e.g. 17, ch. 12], turns out to be a useful precision target and proof method, and our mechanization verifies known exactness results of the literature [18–20]; it also shows that exactness is not preserved by reduced products (§5.5).

We carry out this development as `ROCQ BOTTOM`, under the additional constraint of keeping the trusted assumptions as small as possible: we rely on no axiom beyond Rocq’s core type theory, in particular no excluded middle and no propositional or set extensionality. This serves two purposes. First, it lets us study how far the formalisation of abstract interpretation and optimality proofs can be pushed in a restricted logic. Equivalence arguments that would collapse under extensionality, and that classical presentations of abstract interpretation rely on heavily, must instead be handled explicitly; and the distinctions that emerge (between concrete and abstract orders, between most-precise and best, between bottom and non-bottom representations) turn out to be load-bearing. Second, the resulting library is portable to alternative constructive foundations (e.g., HoTT-like settings) that reject the same axioms. Rocq, in our experience, supports this disciplined, axiom-minimal style better than the alternatives.

The relational and non-extensional style has benefits even outside what is strictly necessary for mechanisation. Take the congruence domain of Granger [19], which represents the set $r + m\mathbb{Z}$ as a pair (r, m) . Some sets have no best abstraction, like the empty set. Some have several: the odd numbers are described equally well by $1 + 2\mathbb{Z}$ and by $3 - 2\mathbb{Z}$, and we may not want to choose a canonical representative. These pathologies prevent the definition of a classical α -function, which requires a single best abstraction for every set, and they make γ non-injective. Yet they are handled cleanly in our development, which reasons explicitly about equivalence relations and uses preorders rather than partial orders. Despite the absence of a classical Galois connection, we recover all the structural properties of a Galois *insertion* — in particular, that the abstract and concrete orders coincide on representable sets — and prove best-abstraction or exactness for the most common transfer functions in this domain (including the truncated and Euclidean divisions).

Contributions Our main contribution is the formalization of the part of standard abstract interpretation theory concerned with precision reasoning, expressed in the Calculus of Inductive Constructions. This formalization is implemented in `ROCQ BOTTOM`, a Rocq library for precision-aware verification of abstract transfer functions which is *axiom-free* and *dependency-free* (outside of Rocq 9.0’s standard library).

- An algebraic hierarchy of abstractions, abstract domains, and abstract lattices, with α -relations replacing total α -functions, and key optional properties (`EXACTORDER`, `JOINISLUB`, γ -injectivity and Galois-insertion) for reasoning about precision (§2).

- A hierarchy of precision relations, such as EXACTLYREPRESENTS, BESTABSTRACTION, and MOSTPRECISE, with mechanised implications, especially when EXACTORDER identifies the abstract and concrete orders (§3).
- Compositional abstract domain and lattices combinators (WithTop, WithBottom, Subset/NonEmpty, Conjunction) that help preserve or establish EXACTORDER, JOINISLUB, α -relations, and the Galois-insertion structure of their components (§4).
- Verified optimal transfer functions in a variety of domains: quadrivalent booleans, known bits, Granger [19] congruences, possibly-unbounded integer intervals, and a reduced product of intervals and congruences, using α -completeness to make some optimality proofs compositional. We systematically verify the best possible precision relation, which is often exactness (§5).
- An axiom-minimal mechanisation: no excluded middle, no propositional or set extensionality, no dependency outside Rocq’s standard library.

The library is young but actively developped, and additional domain combinations and transfer functions are in progress. However, it already supports several nontrivial domains end-to-end and demonstrates that the framework scales through composition.

2 The Algebraic Hierarchy

We implement a hierarchy of algebraic structures mainly following the *packed class* approach of Garillot et al. [21]. This approach is a variant of the bundled approach, which puts algebraic operators and properties in a single data structure that is easy to extend, and does not suffer from the combinatorial explosion of proof terms that occurs when combining different structures in the unbundled approach [22]. This last point is crucial for an abstract interpretation library, as abstract domains are built from modular combinations of other domains [2].

Our hierarchy relies on the notion of *propset* $S : \wp(C)$, which is a predicate $C \rightarrow \text{Prop}$. Inclusion is pointwise implication ($S_1 \subseteq S_2 \triangleq \forall c, S_1(c) \Rightarrow S_2(c)$). As we do not rely on propositional or set extensionality, two propsets with the same elements are not equal, only *equivalent*: $S_1 \sqsubseteq S_2 \triangleq S_1 \subseteq S_2 \wedge S_2 \subseteq S_1$. We use $\sqsubseteq$ everywhere in place of equality on sets.

2.1 Abstraction

The bottom of the hierarchy, which we call an *abstraction*, consists only of a carrier type and a concretization function giving meaning to its elements (and no laws).

**Definition 1 (Abstraction 

- a carrier type A (the type of abstract elements),**

- a concretization function $\gamma_A : A \rightarrow \wp(C)$.

We write γ_A for the concretization of abstraction A .

In making this definition, we have already made several design choices:

- We chose to relate abstract elements to sets, instead of arbitrary structures (e.g., other lattices). Relating an abstract element to a set (i.e., to properties over concrete objects) is the most common use of abstract interpretation, and allowing arbitrary objects would have required another layer of definition that would need to be unfolded in every proof.
- We chose to represent sets as “propsets” (i.e., a function $C \rightarrow \text{Prop}$), and not a “boolset” (a function $C \rightarrow \text{bool}$). In many cases we want to discuss undecidable properties, so requiring abstract elements to relate to boolsets would be restrictive.
- We chose to represent the connection between abstract and concrete elements using a γ function, which is one choice among many [11]. There are multiple reasons for this choice [12]; first, α functions cannot be defined in Rocq without introducing additional axioms, as the type system prevents defining functions from logical objects (like propsets) to computable ones; second, working with γ allows for simpler reasoning, as γ simply decodes each abstract element to its logical meaning, while α performs an additional upper-closure transformation. More simply said, the reason is pragmatic: in static analysis, we use computable abstractions to represent uncomputable properties; and all our computations must be given a meaning (using γ). Starting from the other direction, arbitrary properties may not always have a best abstraction, or it can be difficult to characterize, and we often only want a sufficiently precise one anyway.

The C abstraction definition allows stating many of our precision notions (over- and under-approximation, exact representation, and maximal precision), which we detail in Section 3. In particular, we can already compare the precision of two abstract elements, using the *concrete order*:

Definition 2 (Concrete order ). Given an abstraction A , the concrete order is:

$$a_1 \subseteq_A^\gamma a_2 \triangleq \gamma_A(a_1) \subseteq \gamma_A(a_2)$$

It is easy to prove that this concrete order is a preorder , i.e. that it is reflexive and transitive. It is not antisymmetric, as we may have different abstract elements with the same precision (e.g., the intervals $[3, 2]$ and $[4, 2]$ both represent an empty set). However, elements with the same concretization can still be compared, by introducing an abstract order.

2.2 Abstract Domain: adding an abstract order

Even though the concrete order may be the one in which we are ultimately interested, it suffers from different issues:

- it is generally not computable;
- it fails to represent important nuances in precision; for instance, while `False` and the reduced product between the interval $[2, 5]$ and congruence class $7\mathbb{Z}$ both represent the empty set, the first one is more precise as information that exists in the second has been propagated;
- it lacks some important compositional properties; in particular, being the most-precise element in a product (a, b) representing a conjunction does not imply that either component is the most-precise (Section 4).

One often overlooked aspect of abstract interpretation is the notion of *abstract order*, which is not just a computable over-approximation of the concrete order, but a structural order that solves all these issues. We thus extend abstractions with an *abstract order* $\sqsubseteq_A$, together with laws ensuring it is sound with respect to γ and the concrete order. Abstract interpretation really begins when we have this abstract order (hence, we call this extension an *abstract domain*).

Definition 3 (Abstract domain . A C abstract domain is a C abstraction equipped with:

- an abstract order $\sqsubseteq_A: A \rightarrow A \rightarrow \text{Prop}$,
- a law that $\sqsubseteq_A$ is a preorder,
- a law that γ_A is monotone: $a_1 \sqsubseteq_A a_2 \Rightarrow \gamma_A(a_1) \subseteq \gamma_A(a_2)$.

The monotonicity law is *soundness of the abstract order*: the abstract order implies the concrete one. This can be interpreted either as $\sqsubseteq_A$ being more conservative than $\subseteq_A^\gamma$ (as it can fail to recognize that one element is more precise than another), or conversely, as being more discriminating (insofar as it can distinguish abstract elements that share the same concretization).

Given that $\sqsubseteq_A$ is a preorder, we can derive an equivalence relation from it:

Definition 4 (Abstract equivalence . $a_1 \sqsubseteq_A a_2 \triangleq a_1 \sqsubseteq_A a_2 \wedge a_2 \sqsubseteq_A a_1$.

And we can prove it is an equivalence relation, and that γ_A maps $\sqsubseteq_A$ -equivalent elements to $\subseteq$ -equivalent sets. The benefits of having such an equivalence relation are that we don't need to have canonical representatives of abstract elements with the same precision; for instance, in the congruence domain, the pairs $(2, 3)$ and $(5, -3)$ are equivalent and represent equivalent sets $\{z : \mathbb{Z} \mid \exists k : \mathbb{Z}, z = 2 + 3 * k\} \subseteq \{z : \mathbb{Z} \mid \exists k : \mathbb{Z}, z = 5 - 3 * k\}$. This implies that in the transfer functions, we do not have to make the abstract elements canonical, saving the arithmetic operations that canonicalization performs (e.g., reducing r modulo m in a congruence pair (r, m)). On the other hand, a canonical representation may allow cheaper equivalence tests.

Another use case of this equivalence relation arises in the presence of subset or dependent existential types, which pair an element with a proof of a property. If we don't assume the proof irrelevance axiom, intensional equality is too strong, as it distinguishes between different proofs of the same proposition. Thus, an equivalence relation that identifies pairs using their underlying witnesses, rather than their proofs, provides a more appropriate notion of equality.

When the abstract and concrete orders coincide (i.e., γ is an order-embedding), the theory simplifies considerably:

Definition 5 (EXACTORDER ). An abstract domain A has EXACTORDER if:

$$\forall a_1, a_2, \quad a_1 \sqsubseteq_A a_2 \Leftrightarrow a_1 \subseteq_A^\gamma a_2$$

Much of Section 4 is concerned with making sure that this property holds, e.g. by special-casing elements that represent the empty set. When this property holds, this implies  the weaker notion of γ -injectivity: distinct abstract elements have distinct concretizations (up to the respective equivalences).

Definition 6 (ISGAMMAINJECTIVE ). An abstract domain A is γ -injective if:

$$\forall a_1, a_2, \quad \gamma_A(a_1) \sqsubseteq \gamma_A(a_2) \Rightarrow a_1 \sqsubseteq_A a_2.$$

The converse fails: γ can be injective while still leaving the abstract order strictly finer than the concrete one.

Example 1 (ISGAMMAINJECTIVE does not imply EXACTORDER ). Take the concrete type $C \triangleq \{\bullet\}$ (a singleton), the carrier $A \triangleq \text{bool}$, equality as the abstract order ($a_1 \sqsubseteq_A a_2 \triangleq a_1 = a_2$), and $\gamma_A(\text{true}) \triangleq \{\bullet\}$, $\gamma_A(\text{false}) \triangleq \emptyset$.

Then γ is injective: *true* and *false* have distinct concretizations. But EXACTORDER fails: $\gamma_A(\text{false}) \subseteq \gamma_A(\text{true})$, so $\text{false} \subseteq_A^\gamma \text{true}$ in the concrete order, while $\text{false} \neq \text{true}$ in the abstract order. The deeper reason (Section 2.4) is that the empty set has no best abstraction in this domain—both *true* and *false* over-approximate it, and they are incomparable in the abstract order.

When EXACTORDER does not hold, we gain an opportunity to enhance precision through *reduction*: refining an element (making it smaller in $\sqsubseteq_A$) without changing its meaning, by incorporating newly learned facts. This fundamental operation is ubiquitous in computer science; for instance in constraint programming, it corresponds to constraint propagation [23–25]; in e-graphs, to equality saturation [26, 27].

Definition 7 (REDUCTION ). A function $\text{red} : A \rightarrow A$ is a reduction if:

1. $\forall a, \text{red}(a) \sqsubseteq_A a$ (refines),
2. $\forall a, \gamma_A(\text{red}(a)) \sqsubseteq \gamma_A(a)$ (preserves meaning).

For instance, the basic product of interval and congruence information does not have EXACTORDER: the abstract elements $([2, 10], 3\mathbb{Z})$ and $([3, 9], 3\mathbb{Z})$ concretize to the same set $\{3, 6, 9\}$. This allows reducing the former into the latter: since neither 2 nor 10 are multiples of 3, we can shrink the interval to the nearest multiple of 3. The latter element cannot then be reduced any further, in the sense that no abstract element with the same concretization is strictly below it in $\sqsubseteq_A$. Formally defining this notion requires, though, a notion of *best abstraction of a set*, which is what α -relations capture.

2.3 α -Relations: a Replacement for Galois Connections

Now that we have the notions of concrete and abstract orders, we can express that an abstract element is the best abstraction of a concrete set. As we will see in Section 3, this is equivalent to stating a per-element *adjunction* property between the abstract order and the concrete subset order.

Definition 8 (IsALPHA ). *Given an abstract domain A , an element $\alpha_S : A$ is adjoint to a set $S : \wp(C)$ if:*

$$\forall a : A, S \subseteq \gamma_A(a) \Leftrightarrow \alpha_S \sqsubseteq_A a$$

In the classical Galois connection setting, one defines a function $\alpha : \wp(C) \rightarrow A$, sets $\alpha_S \triangleq \alpha(S)$, and requires the IsALPHA property to hold for every set S . Rocq’s type system prevents defining such a function without additional classical axioms; however, we can instead define less demanding α -relations. The idea is that instead of defining a function $\alpha : \wp(C) \rightarrow A$, we define a relation $\mathcal{R}_\alpha : A \rightarrow \wp(C) \rightarrow \text{Prop}$, with the intent that $\alpha_S \mathcal{R}_\alpha S \Leftrightarrow \alpha_S = \alpha(S)$.

One immediate benefit of this approach is that even in classical settings, the α function cannot always be defined because there isn’t always a best abstraction of every concrete set (e.g., there is no best abstraction of π using rational intervals, or no best abstraction of a circle using polyhedra [28]); but there can still be a best abstraction of some concrete sets (e.g., $[1, 1]$ is the best abstraction of $\{1\}$, and there are best abstractions of a triangle using polyhedra). Another benefit is that the best abstraction does not need to be unique (all potential best abstractions are just $\sqsubseteq_A$ -equivalent). Thus, α relations provide a more general setting for reasoning about precision than α functions.

We actually distinguish two kinds of α -relations:

Definition 9 (Strong  and weak  α -relations). *Given a relation $\mathcal{R}_\alpha : A \rightarrow \wp(C) \rightarrow \text{Prop}$, $\mathcal{R}_\alpha$ is a strong α -relation if*

$$\forall a, S, a \mathcal{R}_\alpha S \Leftrightarrow \text{IsALPHA } a S,$$

and a weak α -relation if

$$\forall a, S, a \mathcal{R}_\alpha S \Rightarrow \text{IsALPHA } a S.$$

A weak α -relation suffices to prove that a given element is the best abstraction of a set. A strong α -relation can also prove that some set does *not* have a best abstraction.

Example 2. Consider the domain of upper bounds over a preorder $(A, \leq)$, where $\gamma_A(a) = \{c \mid c \leq a\}$. In this setting, the relation $\text{is_lub}(a, S)$, stating that a is the least upper bound of S , is a strong α -relation: it characterizes exactly when a is adjoint to S .

To handle sets that lack a finite upper bound in A , we extend the domain with a $\top$ element, representing the absence of an upper bound. While $\top$ is the

best abstraction for any unbounded set, we can only prove this constructively by assuming a sufficient condition: that S is *cofinal* in A (i.e., $\forall z : A, \exists z' \in S, z \leq z'$). This property provides a positive witness to refute any candidate finite bound.

However, cofinality is not a necessary condition; the converse—that $\top$ being the best abstraction implies cofinality—fails in general. Bridging this gap would require both a *total order* (to ensure that an element not below a bound must be above it) and extracting a witness from the logical negation of boundedness ($\neg \forall \rightarrow \exists \neg$), which is not constructively valid, even when the order is total and decidable. Consequently, cofinality only allows defining a *weak* α -relation: it suffices for proving optimality in practice, but it cannot prove that every unbounded set has $\top$ as its best abstraction.

2.4 Galois Insertion and Maximal Reduction

We can now characterize the reductions of Section 2.2 that cannot be improved any further: a has reached this point exactly when a is its own best abstraction.

Definition 10 (MAXIMALLYREDUCED ). An element $a : A$ is maximally reduced if it is a best abstraction of its own concretization:

$$ISALPHA\ a\ (\gamma_A(a)).$$

In the classical Galois-connection setting, this corresponds to $a = \alpha(\gamma_A(a))$, i.e., that the canonical reduction $\alpha \circ \gamma$ leaves a unchanged. In our relational setting we cannot speak of $\alpha(\gamma_A(a))$ as a term, but we can still ask whether a is a best abstraction of $\gamma_A(a)$, and that is what the definition captures.

For instance, in the basic interval-congruence product of Section 2.2, the element $([3, 9], 3\mathbb{Z})$ is maximally reduced: every abstract element that over-approximates $\{3, 6, 9\}$ lies above it in $\sqsubseteq_A$. A function that returns a maximally-reduced element from each input is a *maximal reduction* .

Galois insertion. In the classical formulation, a *Galois insertion* is a Galois connection in which γ is injective. Because the classical setting assumes a full Galois connection—a single best abstraction exists for *every* concrete set—adding γ -injectivity is enough to make every element maximally reduced and the order exact. In our relational setting, best abstractions exist only for *some* sets; γ -injectivity and EXACTORDER therefore come apart (Example 1),

Note that the requirement is much weaker than requiring the existence of an α function: we don’t need a best abstraction for arbitrary sets, only for those corresponding to the concretization of abstract elements. Furthermore, there may be multiple best abstractions.

Indeed, when the domain is a Galois insertion (or more locally, when the arguments to a transfer function are maximally reduced) reasoning about precision often simplifies to a structural argument: best-abstraction proofs compose better. The next subsection makes this concrete with the join operation.

2.5 Join Semilattice and Lattice

We add two further extensions to our algebraic hierarchy: first with join-semilattices, then with lattices. In our examples, we focus on join-semilattices rather than meet-semilattices because many of our abstractions are not closed under intersection: we are often interested in abstractions that cannot represent the empty set, which is a possible outcome of intersection. For instance, the congruence domain [19] represents a set $r + m\mathbb{Z}$ using a pair (r, m) of integers; it cannot represent the empty set, and is thus not closed under meet (the lattice must be extended with a bottom element to represent the empty set). For this reason, our current algebraic hierarchy is linear, i.e. it does not include a meet-semilattice layer.

Definition 11 (Join-semilattice ). A C join-semilattice is a C abstract domain equipped with a join operation $\sqcup_A : A \rightarrow A \rightarrow A$, satisfying the following laws:

$$\forall a_1, a_2 : \quad a_1 \sqsubseteq_A (a_1 \sqcup_A a_2) \quad \wedge \quad a_2 \sqsubseteq_A (a_1 \sqcup_A a_2)$$

The join-semilattice laws, with the monotonicity of γ_A , ensure soundness: we can prove that the concretization of the join over-approximates the union:

$$\gamma_A(a_1) \cup \gamma_A(a_2) \subseteq \gamma_A(a_1 \sqcup_A a_2) \quad (\text{JOINSOUND} \img alt="icon" data-bbox="768 437 783 452/>)$$

The key optional property is JOINISLUB, which states that the join is not just an over-approximation of the union, but the *least* upper bound in the abstract order:

Definition 12 (JOINISLUB ). A join-semilattice A has JOINISLUB if for all a_1, a_2, a' :

$$a_1 \sqsubseteq_A a' \wedge a_2 \sqsubseteq_A a' \Rightarrow (a_1 \sqcup_A a_2) \sqsubseteq_A a'$$

When JOINISLUB holds, we recover the standard algebraic laws of join-semilattices: associativity, commutativity, and idempotency, up to $\sqsubseteq_A$ -equivalence. The *connecting lemma* [29, §2.8] bridges the order-theoretic and algebraic views of join-semilattices:

Lemma 1 (Connecting lemma ). Under JOINISLUB, the following are equivalent:

$$a_1 \sqsubseteq_A a_2 \Leftrightarrow (a_1 \sqcup_A a_2) \sqsubseteq_A a_2$$

The converse direction also holds: if $a_1 \sqsubseteq_A a_2 \Leftrightarrow (a_1 \sqcup_A a_2) \sqsubseteq_A a_2$ is satisfied together with associativity of $\sqcup_A$ and its compatibility with $\sqsubseteq_A$, then JOINISLUB follows . JOINISLUB also enables the following key lemma:

Lemma 2 (Join distributes over α -relations ). Under JOINISLUB, if ISALPHA $a_1 S_1$ and ISALPHA $a_2 S_2$, then ISALPHA $(a_1 \sqcup_A a_2) (S_1 \cup S_2)$.¹

There are three important ways to interpret and use this lemma.

¹ We thank Valentin Perrelle for making us aware of this theorem.

1. it is a compositionality lemma, which is key for proving optimality of many binary transfer functions, where the input space is split and the results joined. This lemma is used, for instance, to prove the optimality of interval multiplication or division, where we split the arguments into positive and negative intervals, to handle the possible sign cases for each operand;
2. the lemma guarantees that $(a_1 \sqcup_A a_2)$ is the `BESTABSTRACTION` of $S_1 \cup S_2$ only when a_1 and a_2 are themselves the `BESTABSTRACTION` of respectively S_1 and S_2 ; in general, $(a_1 \sqcup_A a_2)$ *may not be the best abstraction of the union*. Furthermore, when a_1 and a_2 are not best abstractions, there may be precision losses even in the concrete. Consider for instance the join of the interval-congruence pair $([5, 15], 10\mathbb{Z}) \sqcup_A ([8, 12], 5\mathbb{Z}) = ([5, 15], 5\mathbb{Z})$, whose concretization is $\{5, 10, 15\}$, while both arguments concretize to $\{10\}$, and the best abstraction of $\{10\}$ is $([10, 10], 10 + 0\mathbb{Z})$. Therefore, one should always ensure that the arguments passed to join are maximally reduced (if needed, by calling a reduction operator), as the result is then guaranteed to be the best abstraction of the set-union.
3. because $(a_1 \sqcup_A a_2)$ is the `BESTABSTRACTION` of $S_1 \cup S_2$, it is already `MAXIMALLYREDUCED`; there is no need to call a reduction operator on the result of a join operation. This is another example showing how theoretical insights can help optimize an implementation.

All three observations are in fact instances of a general property of transfer functions: $\sqcup_A$ is our first example of an α -complete transfer function (with set union as its concrete operation), a notion we define in Section 3.2 and allows compositional proofs of transfer functions on a reduced-product (§ 5.5).

Finally, we define *lattices*, which extend a join-semilattice with a meet operation $\sqcap_A$:

Definition 13 (Lattice ). A C lattice is a C join-semilattice equipped with a meet operation $\sqcap_A : A \rightarrow A \rightarrow A$, satisfying the law that meet over-approximates intersection:

$$\gamma_A(a_1) \cap \gamma_A(a_2) \subseteq \gamma_A(a_1 \sqcap_A a_2).$$

We can further extend the hierarchy with additional algebraic laws, but we don't need anything more to prove optimality of transfer functions. The module type for extraction includes additional elements, such as computable equivalence and order relations (returning `bool` instead of `Prop`).

3 Soundness and Precision of Transfer Functions

In this section we first study the possible relations between an abstract element and a set of concrete elements, and then apply this to characterize the soundness and precision of transfer functions.

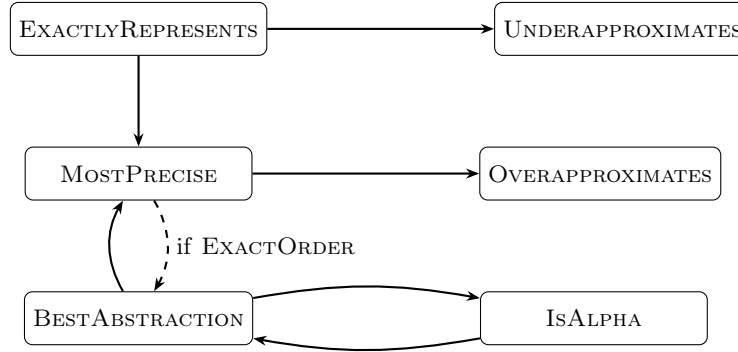


Fig. 1. Implications between precision notions. Solid arrows hold unconditionally. The dashed arrow from MOSTPRECISE to BESTABSTRACTION requires EXACTORDER. This figure explains our recipe for precision proofs: try establishing EXACTORDER, then proving EXACTLYREPRESENTS when feasible and BESTABSTRACTION otherwise.

3.1 Relations between abstract and concrete elements

The heart of abstract interpretation is relating an abstract element with the concrete elements that it represents. When we care only about soundness, we are interested in only one of them: that the abstract element overapproximates the concrete ones. In our framework where we are also interested in precision, we can derive 6 different such relations; 4 of which require only an abstraction (i.e., do not require an abstract order).

Furthermore, there is a hierarchy between the precision notions, where some notions imply others. Figure 1 summarizes these implications: the vertical arrows hold unconditionally; the dashed arrow from MOSTPRECISE to BESTABSTRACTION holds only under EXACTORDER.

Concrete-Precision Notions

Definition 14. *Given a C -abstraction A :*

-  An element $a : A$ OVERAPPROXIMATES a set $S : \wp(C)$ if $S \subseteq \gamma_A(a)$.
-  An element $a : A$ UNDERAPPROXIMATES a set $S : \wp(C)$ if $\gamma_A(a) \subseteq S$.
-  An element $a : A$ EXACTLYREPRESENTS a set $S : \wp(C)$ if $\gamma_A(a) \subseteq\subseteq S$.

EXACTLYREPRESENTS is the conjunction of OVERAPPROXIMATES and UNDERAPPROXIMATES (and thus implies both). It is also called γ -completeness, forward-completeness, or exactness in the literature [30]. It is often thought to be a notion too strong and thus less interesting [e.g. 17, ch. 12]. Exactness results are nevertheless not rare (e.g., interval addition and negation [18], congruence addition and subtraction [19], and bitwise operations on known bits [20, 31]). Our case studies (§ 5) take advantage of this, and prove exactness directly (showing that concrete elements in the result have an antecedent in the input).

Definition 15 (MOSTPRECISE ). *Given an abstraction A , an element $a : A$ is MOSTPRECISE for $S : \wp(C)$ if a OVERAPPROXIMATES S and, among all overapproximations, it has the smallest concretization:*

$$\forall a' : A, S \subseteq \gamma_A(a') \Rightarrow \gamma_A(a) \subseteq \gamma_A(a')$$

MOSTPRECISE applies to abstract elements that are maximally-precise, but still sound, abstractions of S , according to the concrete order $\subseteq_A^\gamma$. It is easy to prove that EXACTLYREPRESENTS implies MOSTPRECISE.

Abstract-Precision Notions When an abstract order $\sqsubseteq_A$ is available, we can define another notion of “best precision”: this time according to the abstract order instead of the concrete order. That is, we want the abstract element that overapproximates S , but is maximally precise according to $\sqsubseteq_A$.

Definition 16 (BESTABSTRACTION ). *Given an abstract domain A , an element $a : A$ is a BESTABSTRACTION of $S : \wp(C)$ if a OVERAPPROXIMATES S and for any a' that OVERAPPROXIMATES S , $a \sqsubseteq_A a'$.*

The last notion has already been defined in Section 2: it is ISALPHA $a S$, defined as $\forall a' : A, S \subseteq \gamma_A(a') \Leftrightarrow a \sqsubseteq_A a'$. Since an abstract domain requires γ_A to be monotone, it is equivalent to BESTABSTRACTION:

Lemma 3.  ISALPHA $a S \Leftrightarrow$ BESTABSTRACTION $a S$.

Some proofs are more easily done using BESTABSTRACTION (for instance, when the structure of the proof mirrors the definition of BESTABSTRACTION), while others are more natural using ISALPHA.

Finally, as we explained in Section 2, the abstract order can be seen as finer-grained than the concrete order, which makes BESTABSTRACTION a stronger requirement than MOSTPRECISE: being the smallest in the abstract order implies being the smallest in the concrete order, but not vice versa.

Lemma 4.  BESTABSTRACTION $a S \Rightarrow$ MOSTPRECISE $a S$.

The proof follows directly from the definition of BESTABSTRACTION and the fact that γ_A is monotone. If we have EXACTORDER, we have the converse implication:

Lemma 5.  Under EXACTORDER, MOSTPRECISE $a S \Leftrightarrow$ BESTABSTRACTION $a S$.

The proof follows from the fact that γ_A is order-embedding. Without EXACTORDER, MOSTPRECISE does *not* imply ISALPHA. For instance, in the interval domain, both $[8, 3]$ and $[9, 2]$ most-precisely represent the empty set (their concretizations are both empty), but $[9, 2] \sqsubseteq_A [8, 3]$ (the interval shrinks), so $[8, 3]$ is not a best abstraction of $\emptyset$.

The Hierarchy of Implications If we examine the full implication diagram, we see that there are three methods to prove that an abstract element a is the most precise abstraction of S : we can either

- prove that it EXACTLYREPRESENTS S (for sets that can be represented exactly by an abstract element);
- or that it is the BESTABSTRACTION of S (which requires the definition of an abstract order);
- or directly prove that it is the MOSTPRECISE abstraction of S (but this is a weaker property than the others).

If the abstraction does not have the EXACTORDER property (like the product between interval and congruences), then all these properties are different. For instance, the set $\{2, 4, 6, 8\}$ is exactly represented by the product $[1, 9] \cap 2 * \mathbb{Z}$, which is also a most precise abstraction, but is not the best (the best is $[2, 8] \cap 2 * \mathbb{Z}$). The set $\{2, 4, 8\}$ admits $[2, 8] \cap 2 * \mathbb{Z}$ as its best (and most-precise) abstraction, but cannot be exactly represented. Finally, $[1, 9] \cap 2 * \mathbb{Z}$ is a most-precise abstraction of the set $\{2, 4, 8\}$, but it is neither the best nor an exact representation of this set.

If the abstraction has the EXACTORDER property (which is the case for many abstractions, like congruences and non-empty intervals, or if we only consider maximally-reduced elements in the product of intervals and congruences), then EXACTLYREPRESENTS is the strongest property, and the other two are equivalent. For instance, the set $\{4, 5, 6, 7\}$ can be exactly represented by the interval $[4, 7]$, which implies that $[4, 7]$ is also the best abstraction and the most-precise abstraction of this set. The set $\{4, 6, 7\}$ cannot be exactly represented by the interval $[4, 7]$, but $[4, 7]$ is nevertheless the best and most-precise abstraction of that set.

Thus, the ideal method is to make sure that our abstraction has the EXACTORDER property. This can be achieved by systematically performing a maximal reduction to return the best (in the abstract) element whenever we would return a most-precise one, or more generally by not considering problematic abstract elements, like the multiple representations of the empty set. Once EXACTORDER is established, the proof strategy is to prove EXACTLYREPRESENTS when feasible, and BESTABSTRACTION otherwise.

3.2 Application to the specification of transfer functions

We can now use these notions to reason about the precision of transfer functions (instead of merely sets). First, we need to lift concrete operations to operations on sets.

Definition 17 (Lifting functions). Given $f : C_1 \rightarrow C_0$, we define:

- the forward-lift of f : $\text{fwd}(f)(S_1) = \{c_0 \mid \exists c_1 \in S_1, f(c_1) = c_0\}$
- the backward-lift of f : $\text{bwd}(f)(S_1, S_0) = \{c_1 \in S_1 \mid f(c_1) \in S_0\}$

Given $f : C_2 \rightarrow C_1 \rightarrow C_0$, we define:

- the forward-lift: $\text{fwd}_2(f)(S_2, S_1) = \{c_0 \mid \exists c_2 \in S_2, \exists c_1 \in S_1, f(c_2, c_1) = c_0\}$
- the guarded forward-lift, given a predicate $P : C_2 \rightarrow C_1 \rightarrow \text{Prop}$:

$$\text{fwd}_2^P(f)(S_2, S_1) = \{c_0 \mid \exists c_2 \in S_2, \exists c_1 \in S_1, P(c_2, c_1) \wedge f(c_2, c_1) = c_0\}$$

- the left and right backward lifts:

$$\text{bwd}_L(f)(S_2, S_1, S_0) = \{c_2 \in S_2 \mid \exists c_1 \in S_1, \exists c_0 \in S_0, f(c_2, c_1) = c_0\}$$

$$\text{bwd}_R(f)(S_2, S_1, S_0) = \{c_1 \in S_1 \mid \exists c_2 \in S_2, \exists c_0 \in S_0, f(c_2, c_1) = c_0\}$$

These lift operations are standard [18]; forward lifts are used to define the computation done when analysing expressions in forward static analyses, while backward lifts define how one refines the possible values of expressions after a conditional statement. The guarded (forward) operation is useful for partial operations such as division and quotient, which Rocq totalizes. Without the guard, the collecting semantics would include the results of dividing by zero, causing an unnecessary loss of precision.

Transfer Function Specifications Given a precision relation $\mathcal{R}$ between abstract elements and concrete sets (such as OVERAPPROXIMATES, MOSTPRECISE, BESTABSTRACTION, or EXACTLYREPRESENTS), we can specify the precision of a transfer function by requiring that the relation holds between the abstract output and the output of the collecting semantics of the concrete operation.

Definition 18 (Transfer function specification). *Given a precision relation $\mathcal{R}$, an abstract n -ary function $f_A : A_n \rightarrow \dots \rightarrow A_1 \rightarrow A_0$ satisfies $\mathcal{R}$ with respect to a concrete operation $f_C : \wp(C_n) \rightarrow \dots \rightarrow \wp(C_1) \rightarrow \wp(C_0)$ on sets if $\mathcal{R}$ holds between the abstract output and the concrete output for every tuple of inputs:*

$$\forall a_n \dots a_1, (f_A a_n \dots a_1) \mathcal{R} (f_C (\gamma_{A_n}(a_n)) \dots (\gamma_{A_1}(a_1)))$$

We instantiate this for $n = 1, 2, 3$. Note that the functions can be multi-sorted; e.g., transfer functions for $\text{fwd}_2(\text{Z.leb})$ takes integer abstraction and return a boolean abstraction; transfer functions for $\text{fwd}_2(\text{Z.div})$ take non-empty integer abstraction and return a possibly-empty integer abstraction.

We say f_A is *sound* if it satisfies OVERAPPROXIMATES, *most precise* if it satisfies MOSTPRECISE, *exact* if it satisfies EXACTLYREPRESENTS, and *best* if it satisfies BESTABSTRACTION (which requires all domains to be abstract domains, not just abstractions).

One further specification is not obtained this way from a precision relation, because it strengthens the *input* side: it considers arbitrary sets as inputs, instead of only those that correspond to the concretization of an abstract input.

Definition 19 (α -completeness). *A transfer function f_A is α -complete at abstract inputs $a_n, \dots, a_1$ and concrete sets $S_n, \dots, S_1$ if*

$$\text{ISALPHA } a_n S_n \wedge \dots \wedge \text{ISALPHA } a_1 S_1 \Rightarrow \text{ISALPHA } (f_A a_n \dots a_1) (f_C S_n \dots S_1).$$

This is the backward-completeness equation $f_A \circ \alpha = \alpha \circ f_C$ of the classical setting [30], stated relationally and per input tuple. A best transfer function is only constrained against the concretizations of its inputs; an α -complete one is constrained against all the input sets that its inputs abstract best. Maximal reduction (Section 2.4) bridges the two notions: when every input is maximally reduced, α -completeness at the concretizations yields **BESTABSTRACTION** of the output , and under **EXACTORDER**, α -completeness at all concretization tuples implies that the transfer function is best . The reverse implication does not hold: a *best* transfer function need not be α -complete, because abstracting an input set S_i into a_i may already lose information that would have made the result more precise; known-bits addition is a proved counterexample (§5.5). α -completeness generalizes Lemma 2, which states precisely that $\sqcup$ is α -complete for $\cup$ — in classical terms, that α , being a left adjoint, preserves least upper bounds whenever the join is one. The three observations made after that lemma hold for any α -complete transfer function: its outputs are best abstractions, hence maximally reduced and never in need of a reduction step ; it guarantees nothing when its inputs are not best abstractions, which is why arguments should be kept maximally reduced; and it makes optimality proofs *compose*: through joins for case splits, and componentwise through reduced products (Section 5.5).

In practice, we will initially prove soundness when we only want the analysis to be correct; to prove that a transfer function is best, we can either prove that the transfer function is exact when possible and when we have **EXACTORDER**; or that a function is α -complete when the arguments are **MAXIMALLYREDUCED**; or prove directly that a function is best when neither of these incomparable $[?]$ properties hold. Note that whether a transfer function can achieve exactness or α -completeness is independent of its definition; it is rather an intrinsic property of how well the abstract domain aligns with the concrete operation (e.g., the known-bit abstraction aligns well with bitwise operations, while congruence and interval operations align well with additions and subtractions).

4 Abstraction Combinations

Non-trivial abstract domains are rarely built from scratch; instead, they are *assembled* from simpler pieces [2]. In this section, we describe how our formal framework allows composing abstractions, abstract domains, semi-lattices and lattices. The composition patterns carry the proofs, not just the definitions: once a property is established on the components, the assembled domain inherits it.

A recurring concern across the section is building a Galois insertion (Section 2.4): i.e., defining α -relation predicates, and preserving **EXACTORDER**. This allows proving transfer functions exact when feasible, which is stronger than best in that case. We will see **EXACTORDER** come up at every combinator: required as a side condition for **WithTop**, lost by **Conjunction**, and recovered by two families of *shaping* combinators—one that restricts the carrier (built around **Subset**, Section 4.4) and one that coarsens the abstract order (Section 4.5).

To make this concrete, we use the integer interval domain as a running example throughout the section. The starting point are the bound abstractions $\text{GLB } \mathbb{Z}$ and $\text{LUB } \mathbb{Z}$ — themselves obtained as a parametric construction over a partial order, developed in Section 4.1 below. Each abstracts a set of integers by its greatest lower bound, respectively least upper bound, and is a lattice with `EXACTORDER` and a strong α -relation, but cannot represent unbounded sets. From these we assemble intervals through a chain of combinators:

$$\begin{aligned}
 \text{glbtop} &\triangleq \text{WithTop}(\text{GLB } \mathbb{Z}) & \text{lubtop} &\triangleq \text{WithTop}(\text{LUB } \mathbb{Z}) \\
 \text{itv} &\triangleq \text{Conjunction}(\text{glbtop}, \text{lubtop}) \\
 \text{nbitv} &\triangleq \text{NonEmpty}(\text{itv}) \\
 \text{itv_collapsed} &\triangleq \text{CollapsedBottom}(\text{itv}) \\
 \text{itv_canon} &\triangleq \text{CanonicalBottom}(\text{itv}, \perp)
 \end{aligned}$$

`WithTop` adds a $\top$ element to each bound, representing the absence of a known bound — or, equivalently, a $-\infty$ (resp. $+\infty$) bound. `Conjunction` pairs them so that an interval is the intersection of a lower-bound constraint and an upper-bound constraint. The constructions on the first two lines yield a join-semilattice with `JOINISLUB`, but on `itv` alone the abstract order is not exact: every pair (l, h) with $l > h$ concretizes to $\emptyset$, yielding many syntactically distinct $\sqsubseteq$ -incomparable representations of bottom.

The three constructions on the bottom row are alternative shapings of `itv` that recover `EXACTORDER`, hence a Galois insertion; they differ in how they treat γ -empty representations and in the operational price paid to detect them. `nbitv` *discards* every empty representation, and is the natural target for transfer functions on total operations. `itv_collapsed` keeps the carrier of `itv` unchanged and *collapses* all γ -empty elements into a single $\sqsubseteq_A$ -equivalence class below every non-empty interval, requiring a domain-specific `non_bottom` predicate to recognise them. `itv_canon`, in contrast, restricts the carrier to a *single chosen* $\perp$ representative (plus all non-empty intervals) and coarsens the order so that $\perp$ is the minimum, replacing the emptiness check with a constant-time equality $a = \perp$. A fourth, fully generic shaping combinator, `MaximallyReducedSubset`, restricts the carrier to maximally-reduced elements; we apply it to the reduced product of intervals and congruences in Section 5.5.

4.1 Bound abstractions: GLB, LUB

The first abstraction we encounter is itself parametric. A *concrete lattice*  is a preorder $(A, \leq)$ equipped with $\min, \max : A \rightarrow A \rightarrow A$ and an equivalence $\equiv$, with $\min$ a glb, $\max$ an lub, and $\equiv$ the symmetric closure of $\leq$. The integers $(\mathbb{Z}, \leq, \min, \max, =)$ form one such concrete lattice; so do finite sets under $\subseteq$.

The `GLB` module  defines

$$\gamma_{\text{GLB } A}(a) = \{c \in A \mid a \leq c\} \quad a_1 \sqsubseteq_{\text{GLB } A} a_2 \triangleq a_2 \leq a_1.$$

Table 1. Summary of the abstraction combinators used in this section. For each combinator, the carrier of the assembled domain, its concretization, the lifted α -relation, and whether EXACTORDER and JOINISLUB are preserved by the construction.

Combinator	Carrier / γ	EXACTORDER α -relation	JOINISLUB
WithTop A 	Top NotTop A $\gamma_A(a), C$ on $\top$	if A unbounded below  $a \mathcal{R}_\alpha^A S \Rightarrow (\text{NotTop } a) \mathcal{R}_\alpha S$ $is_ \top_best S \Rightarrow \text{Top } \mathcal{R}_\alpha S$	preserved 
Conjunction (A, B) 	$A \times B$ $\gamma_A(a) \cap \gamma_B(b)$	not in general $(a, b) \mathcal{R}_\alpha S \Leftrightarrow a \mathcal{R}_\alpha^A S \wedge b \mathcal{R}_\alpha^B S$	if both 
Subset (A, P) 	$\{a : A \mid P a\}$ $\gamma_A(a)$	inherited  $\mathcal{R}_\alpha^A$	if P closed under $\sqcup$ 
NonEmpty A 	$\{a : A \mid \exists c, c \in \gamma_A(a)\}$ $\gamma_A(a)$	recovered (e.g. itv)  $\mathcal{R}_\alpha^A$	inherited (via Subset)
MaximallyReducedSubset A 	$\{a : A \mid \text{MAXIMALLYREDUCED}(a)\}$ $\gamma_A(a)$	unconditional  $\mathcal{R}_\alpha^A$	if JOINISLUB A 
CollapsedBottom A 	A unchanged $\gamma_A(a)$	if dec. γ -emp. + EXACTORDER on non-empty  $\exists c, c \in S \wedge a \mathcal{R}_\alpha^A S \Rightarrow a \mathcal{R}_\alpha S$ $S \subseteq \emptyset \wedge \text{-non_bottom}(a) \Rightarrow a \mathcal{R}_\alpha S$	if dec. γ -emp., via redefined join 
CanonicalBottom $(A, \perp)$ 	$\{a : A \mid \text{non_bottom}(a) \vee a = \perp\}$ $\gamma_A(a)$	via ad_lub  $\mathcal{R}_\alpha^A$ (order coarsened so $\perp$ is min; join redefined: $\sqcup \perp x = x$)	via ajsl_lub 

GLB A has EXACTORDER and a strong α -relation given by is_glb , the fact that an element is the greatest lower bound of a set; its lattice takes $\sqcup = \min$, $\sqcap = \max$, with JOINSLUB immediate from $\min$ being a glb . The dual LUB is GLB applied to the flipped preorder; every property transports.

Instantiated on $\mathbb{Z}$, **GLB** $\mathbb{Z}$ and **LUB** $\mathbb{Z}$ are both lattices with EXACTORDER and strong α -relations, but cannot represent $\mathbb{Z}$ itself. The next combinator fixes that.

4.2 WithTop: adjoining a top element

The `WithTop` module  takes an abstraction (or domain, or lattice) over C and adjoins a fresh $\top$ element representing the full concrete set:

$$\begin{aligned} \text{with_top } A &\triangleq \text{Top} \mid \text{NotTop } A \\ \gamma_{\text{WithTop } A}(\text{Top}) &= C & \gamma_{\text{WithTop } A}(\text{NotTop } a) &= \gamma_A(a). \end{aligned}$$

The abstract order places $\top$ above everything, and the lattice operations are extended pointwise via a generic combinator $\boxplus$: $\top$ absorbs joins and is neutral for meets. The construction lifts at every level of the hierarchy $\boxtimes \boxplus \boxtimes$, and the JOINISLUB property is preserved $\boxtimes$.

The interesting question is the α -relation. The element $\text{NotTop } a$ inherits the base relation directly. For Top , however, we cannot in general decide whether $\top$ is the best abstraction of an arbitrary S : we need a side-condition $P : \wp(C) \rightarrow \mathbf{Prop}$ that captures $\text{ISALPHA Top } S$. The lifted relation is then:

$$\mathcal{R}_\alpha^{\text{WithTop}}(\text{Top}, S) \triangleq P(S), \quad \mathcal{R}_\alpha^{\text{WithTop}}(\text{NotTop } a, S) \triangleq \mathcal{R}_\alpha^A(a, S).$$

Whether P delivers a strong or weak α -relation depends on whether P characterizes $\top$'s optimality exactly or only sufficiently. The canonical instance is the cofinality example of Section 2: the fact that one can always find an arbitrarily

small (resp. large) element in a set S is only a sufficient (but very convenient) condition for proving that the best bound for S is $\top$; we thus obtain only a weak α -relation. 

EXACTORDER on **WithTop** A holds whenever the underlying preorder A is unbounded (below, for the GLB variant): if instead A had a global lower bound a , then **NotTop** a and **Top** would have the same concretization without being equivalent in the abstract. Obviously, $\mathbb{Z}$ satisfies this.

Dual variant. **WithBottom**  is the symmetric construction adjoining a $\perp$ element. We can use it when the base domain cannot represent the empty set on its own, as is the case for congruences (Section 5.3). For our running interval example the empty set is already representable inside the conjunction—every pair (l, h) with $l > h$ has an empty concretization, so we don't need it; we use instead the dedicated bottom combinators presented in Section 4.4.

4.3 Conjunction: product of two abstractions

The **Conjunction** module  combines two abstractions A, B of the *same* concrete type C into a pair $A \wedge B$, defines concretization as the conjunction, and abstract order componentwise:

$$\gamma_{A \wedge B}(a, b) = \gamma_A(a) \cap \gamma_B(b), \quad (a_1, b_1) \sqsubseteq_{A \wedge B} (a_2, b_2) \triangleq a_1 \sqsubseteq_A a_2 \wedge b_1 \sqsubseteq_B b_2.$$

The lattice operations are also componentwise, and JOINISLUB lifts whenever both components have it . So far this is unremarkable; the substantive content lies in the precision behavior, which exposes a structural reason to favor abstract precision (BESTABSTRACTION/ISALPHA) over concrete precision (MOSTPRECISE).

Lemma 6 (BESTABSTRACTION is compositional ). *(a, b) is a BESTABSTRACTION of S in $A \wedge B$ if and only if a is a BESTABSTRACTION of S in A and b is a BESTABSTRACTION of S in B . The same statement under ISALPHA is an immediate corollary, via the ISALPHA $\Leftrightarrow$ BESTABSTRACTION equivalence of Section 3 .*

Lemma 7 (MOSTPRECISE is not compositional ). *If a is a MOSTPRECISE abstraction of S in A and b is a MOSTPRECISE abstraction of S in B , then (a, b) is a MOSTPRECISE abstraction of S in $A \wedge B$. However, there exist A, B, S and $(a, b) \in A \wedge B$ such that (a, b) is MOSTPRECISE for S , while neither a is MOSTPRECISE for S in A nor b is MOSTPRECISE for S in B .*

The core reason for this difference is that the abstract order is defined component-wise; it has thus a strong structure where a pair (a, b) is the smallest element if, and only if, each of a and b is the smallest. On the contrary, the concrete order does not have this structure; it is only a combination of independent facts. We can thus find counter-examples where the combination is precise but each element is not; for instance the pair $([1, 5], 0 + 3\mathbb{Z})$ is MOSTPRECISE for (actually,

it even EXACTLYREPRESENTS) the set $\{3\}$, but each individual component is not the MOSTPRECISE abstraction (the interval $[3, 3]$ and the congruence $3 + 0\mathbb{Z}$ have a more precise concretization). This problem does not happen for BESTABSTRACTION, as $([1, 5], 0 + 3\mathbb{Z})$ is not a BESTABSTRACTION of $\{3\}$.

This asymmetry is one of the structural reasons why the algebraic hierarchy in Section 2 is built around the abstract order: BESTABSTRACTION composes through products, MOSTPRECISE does not.

Application to intervals. In our running example, `itv` = `Conjunction(glbtop, lubtop)` inherits JOINISLUB from both components and admits compositional best-abstraction. Crucially, however, it loses EXACTORDER: the empty set has many incomparable witnesses (every (l, h) with $l > h$), so γ is no longer injective and the conjunction is no longer a Galois insertion. The next subsection presents the family of `Subset`-based shaping combinators that recover it.

4.4 Subset and shaping combinators toward a Galois insertion

Given an abstraction A and a predicate $P : A \rightarrow \text{Prop}$, the `Subset` module forms the dependent-pair carrier $\{a : A \mid P a\}$, with γ and $\sqsubseteq_A$ inherited from the base. The abstraction and abstract-domain levels lift unconditionally; the join-semilattice and lattice lift only when P is closed under the corresponding operation (the user supplies closure proofs). The effect on α -relations is asymmetric: a best abstraction of S in the base domain that does not satisfy P is lost, but every best abstraction that satisfies P remains one in the restricted domain, since the order is unchanged. In practice, P is chosen so that no useful best abstractions are discarded.

Some uses of `Subset` are domain-specific, like restricting to canonical representatives (e.g. congruences $r + m\mathbb{Z}$ with $0 \leq r < m$, Section 5.3, to speed up equivalence tests). The other uses are generic recipes for shaping a domain into a Galois insertion: each picks a predicate P such that the restricted carrier has EXACTORDER, hence γ -injectivity. We present two such recipes below; a third one, `CanonicalBottom`, that also requires changing the abstract order, is explained in Section 4.5.

NonEmpty: discard γ -empty elements. `NonEmpty` instantiates `Subset` with the predicate “ $\exists c. c \in \gamma_A(a)$ ”. Applied to our running example, `nbItv` = `NonEmpty(itv)` removes every empty interval; EXACTORDER then holds on the remainder. The α -relation is the base relation restricted to the surviving elements: every BESTABSTRACTION of a non-empty S in A that satisfies the predicate is preserved, while best abstractions of $\emptyset$ are discarded along with the empty representatives.

Lemma 8 (EXACTORDER on non-empty intervals). *Let $[l_1, h_1], [l_2, h_2]$ be two non-empty intervals over an unbounded concrete lattice A . If $\gamma[l_2, h_2] \subseteq \gamma[l_1, h_1]$ then $[l_2, h_2] \sqsubseteq [l_1, h_1]$ in `NonEmpty(IntervalUnbounded A)`.*

The proof is a transitivity argument constructing $\min(c, w)$ or $\max(c, w)$, where w is the non-emptiness witness and c an element of the bound’s concretization; it works for any non-empty interval over any partial order. The defining predicate is purely semantic and not in general decidable, but for many domains it admits a decidable syntactic equivalent: for intervals, $\text{non_bottom}[l, h] \triangleq l \leq h$ (extended to the **Top** cases)  is equivalent to non-emptiness , and provides a constant-time emptiness check. **NonEmpty** is the natural target for transfer functions on total operations: an analyser can skip the analysis of dead expressions (whose inputs are empty), and a generic lemma proves non-emptiness of the result of sound transfer functions of total operations .

MaximallyReducedSubset: restrict to the image of α . **MaximallyReducedSubset**  instantiates **Subset** with $\text{MAXIMALLYREDUCED}(a) \triangleq \text{BESTABSTRACTION}(a, \gamma_A(a))$. **EXACTORDER** then holds unconditionally : if $\gamma_A(a_1) \subseteq \gamma_A(a_2)$, optimality of a_1 forces $a_1 \sqsubseteq_A a_2$. Closure of **MAXIMALLYREDUCED** under $\sqcup$ follows from **ISALPHA** $a_i S_i \Rightarrow \text{ISALPHA}(a_1 \sqcup a_2) (S_1 \cup S_2)$ when the underlying domain has **JOINISLUB**, so **JOINISLUB** lifts too . This is the framework’s incarnation of the textbook Galois insertion: **MAXIMALLYREDUCED**(a) holds exactly when $a \text{ISALPHA} \gamma_A(a)$, i.e. if $a = \alpha(\gamma_A(a))$ (if we could define an α function), so the restriction is the standard “restrict to $\text{Im}(\alpha)$ ” move—but *without* requiring a constructive α on A . We use it to recover **EXACTORDER** on the unrestricted product of intervals and congruences (Section 5.5).

4.5 Coarsening the order to recover a Galois insertion

Operations like division, meet, or the reduced product must be able to return an abstract element representing $\emptyset$, so the **Subset**-based discarding of **NonEmpty** is unavailable to them. The two combinators below take the opposite route: they keep γ -empty elements in the carrier but coarsen the abstract order so that all such elements are equivalent. This gives back **EXACTORDER** (hence a Galois insertion), at the price of two adjustments. First, the α -relation must be extended along the new order: every γ -empty a becomes a **BESTABSTRACTION** of $\emptyset$. Second, widening the order breaks the underlying join (the example below makes this concrete); the join must be redefined accordingly.

CollapsedBottom: collapse all $\perp$ s in the order. **CollapsedBottom**  keeps the carrier and γ of A and coarsens the order so that every γ -empty element lies below everything:

$$a_1 \sqsubseteq_{\text{CollapsedBottom } A} a_2 \triangleq \neg \text{non_bottom}(a_1) \vee a_1 \sqsubseteq_A a_2.$$

All γ -empty elements become $\sqsubseteq$ -equivalent, so **EXACTORDER** on non-empty elements lifts unchanged and the lifted α -relation (Table 1) makes every γ -empty a a **BESTABSTRACTION** of $\emptyset$. The join breakage anticipated above is concrete in **IntervalUnbounded**: $(1, 0)$ is empty, so $(1, 0) \sqsubseteq (2, 3)$ and $(2, 3) \sqsubseteq (2, 3)$, yet $(1, 0) \sqcup (2, 3) = (1, 3) \not\sqsubseteq_A (2, 3)$. The **JOINISLUB**-preserving **fix**  makes γ -empty elements identities for join; this requires a decidable γ -emptiness predicate.

CanonicalBottom: pick one $\perp$ as the minimum. $\text{CanonicalBottom}(A, \perp)$, given a chosen $\perp : A$ with $\gamma_A(\perp) \subseteq \emptyset$, restricts the carrier via **Subset** to a single γ -empty representative $\{a : A \mid \text{non_bottom}(a) \vee a = \perp\}$ and coarsens the order to $a_1 \sqsubseteq a_2 \triangleq a_1 = \perp \vee a_1 \sqsubseteq_A a_2$. Compared to **CollapsedBottom**, $\perp$ is now detected by a constant-time equality $a = \perp$ instead of a domain-specific emptiness predicate. The redefined join is $\perp \sqcup x = x$, $x \sqcup \perp = x$, otherwise $\sqcup_A$; **JOINISLUB** lifts purely from **JOINISLUB** on A . The α -relation makes the chosen $\perp$ the canonical **BESTABSTRACTION** of $\emptyset$, the only γ -empty element that survived the carrier restriction. For intervals we take $\perp \triangleq (\text{NotTop } 1, \text{NotTop } 0)$ and obtain $\text{itv_canon} \triangleq \text{CanonicalBottom}(\text{itv}, \perp)$; the same construction can be used for reduced product between intervals and congruences Section 5.5.

5 Case Studies

We instantiate the framework on four concrete domains: the quadrivalent abstraction **qv** of the booleans, the integer known-bits **kb** (a pointwise lifting of **qv**), the Granger-style integer congruence **cong**, the integer interval domain **itv**, and a reduced product between intervals and congruences. We make no claim of exhaustive coverage on any of these domains; the goal was rather to show that the framework, the combinators of Section 4, and the precision hierarchy of Section 3 are all in place to verify soundness and precision of complex transfer functions. Our case study yields other important insights: optimality proofs guide efficient implementations; exactness of individual operations is frequent enough to be useful as a proof method, although it can be lost by combination (§5.5); α -completeness makes optimality proofs compositional; and LLM-assisted synthesis of proofs and transfer functions was useful but showed limitations.

The development comprises about 8 000 lines of Rocq 9.0.1 across 15 files, with roughly 3.4 K lines of definitions and statements, 4.6 K of proofs, and 2.3 K of comments. The framework itself (algebraic hierarchy, precision relations, combinators, bound abstractions) accounts for about 1.3 K spec and 1.0 K proof; the four case-study domains contribute the rest, with **cong** (0.4/1.6 K spec/proof) and **itv** (0.9/1.2 K) dominating. The development uses no axiom and no library outside Rocq’s standard library.

5.1 Quadrivalent: exact by computation

The quadrivalent domain qv abstracts the booleans by powerset: $\text{qv} \cong \wp(\text{bool})$. Its four elements $\{\perp, \text{QFalse}, \text{QTrue}, \top\}$ correspond to the four subsets of $\{\text{true}, \text{false}\}$, with $\gamma(\perp) = \emptyset$ and $\gamma(\top) = \text{bool}$. The name reflects a four-valued logic where $\perp$ means “neither” and $\top$ means “both”.

A Galois isomorphism, not just an insertion. Because γ is a *bijection* onto $\wp(\text{bool})$, every concrete subset has a unique representative and the framework’s shaping combinators are not needed here: **EXACTORDER**, **JOINISLUB**, the strong α -relation, and $\perp$ all hold primitively. Crucially, the bijection forces *every* boolean

operation to admit an EXACTLYREPRESENTS abstraction by construction — there is no precision to lose.

Verified transfer functions. Forward and backward transfer functions for `not`, `and`, `or`, `xor` are all *exact*; so are $\sqcup$, $\sqcap$ against $\cup$, $\cap$. Each is a constant-time truth table, and the proofs are one-liners via the `AutoReflect` reflection mechanism (§5.6).

5.2 Known bits: pointwise lifting of Quadrivalent

The known-bits domain [32–36] (also known as the bitfield [31] or tristate [20] domain) represents, for each bit position of a set of integers, the possible values of this bit, as a quadrivalent element. We use the two-integer representation $(\text{must0}, \text{must1}) \in \mathbb{Z}^2$, where the i th bit of `must0` (resp. `must1`) is 0 (resp. 1) if the i th bit of the integer must be 0 (resp. 1). It is thus a **Conjunction** of two independent properties.

$$\gamma_{\text{kb}}(m_0, m_1) = \{v \in \mathbb{Z} \mid \forall i, \text{testbit } m_0 \ i = 0 \Rightarrow \text{testbit } v \ i = 0\} \\ \cap \{v \in \mathbb{Z} \mid \forall i, \text{testbit } m_1 \ i = 1 \Rightarrow \text{testbit } v \ i = 1\}$$

This particular encoding has other nice properties; when (m_0, m_1) does not represent an empty set, the represented integers v are such that $m_1 \preceq v \preceq m_0$, where $\preceq$ is the pointwise lifting of the integer ordering in $\{0, 1\}$; bitwise transfer functions and join/meet reduce directly to integer `Z.lor`/`Z.land` operations; the unknown-bit set is recovered quickly as `must0` $\oplus$ `must1`.

Galois insertion. `kb` alone fails EXACTORDER: any (m_0, m_1) pair such that the i th bit has no possible value concretises to the empty set, so empty sets have many incomparable representations. We restrict via `nbkb` $\triangleq$ `NonEmpty(kb)` (§4.4) to get a Galois insertion: then, γ is injective, the abstraction has EXACTORDER, JOINISLUB holds, and the per-bit α -relation is strong (inherited pointwise from `qv`). To recover an explicit $\perp$ for partial operations, one can wrap it with `WithBottom` or `CanonicalBottom` as needed (we have not yet exercised this on `kb`).

Verified transfer functions. The bitwise transfer functions — bitwise `and`, `or`, `xor`, as well as $\sqcap$ and $\sqcup$, which are also bitwise operations — are all *exact*; this is obtained by pointwise lifting from `qv`. Arithmetic is where exactness stops. We have verified that the carry-propagating addition `kb_add` (the `tnum_add` of Vishwanathan et al. [20]) is *best* on non-empty operands, thereby mechanizing their paper-proof optimality result, but neither *exact* (when the two low bits of an operand are unknown, the collecting sum $\{0, \dots, 6\}$ is not representable: any enclosing element also contains 7) nor α -complete, with consequences for reduced products (§5.5). Other arithmetic transfer functions are future work.

5.3 Congruence: a non-canonical join semilattice

The congruence domain [19] abstracts a set of integers using a congruence relation, defined by a pair $(r, m) \in \mathbb{Z}^2$ of a remainder and a modulus. The concretization is $\gamma(r, m) = r + m\mathbb{Z}$, or more precisely $\gamma(r, m) = \{z \in \mathbb{Z} \mid (m \mid z - r)\}$ , where $x \mid y$ is defined by Rocq as $\exists z, y = z * x$ (so that $0 \mid x \Leftrightarrow x = 0$). The preorder $(r_1, m_1) \sqsubseteq (r_2, m_2) \triangleq m_2 \mid m_1 \wedge m_2 \mid (r_1 - r_2)$  makes $(0, 1)$ maximal and the constants $(r, 0)$ minimal.

Non-canonical by design. The pair representation is intentionally non-canonical: (r, m) , $(r + m, m)$, and $(r, -m)$ all denote the same class. Two abstract elements are $\sqsubseteq$ -equivalent precisely when their moduli divide each other and their residues align modulo the common modulus:

$$(r_1, m_1) \sqsubseteq (r_2, m_2) \iff m_1 \mid m_2 \wedge m_2 \mid m_1 \wedge m_2 \mid (r_1 - r_2),$$

i.e. $|m_1| = |m_2|$ and $r_1 \equiv r_2 \pmod{|m_1|}$. We deliberately do *not* quotient the carrier: working modulo $\sqsubseteq$ (which is a proper morphism for γ , $\sqcup$, and the transfer functions) is more natural than choosing a normal form, and lets transfer functions return whichever representative their algorithm produces. If a normal form is wanted (e.g., to test equivalence by syntactic equality), it is recovered as a derived domain via the **Subset** combinator with predicate $m \geq 0 \wedge (m = 0 \vee 0 \leq r < m)$.

Galois insertion. The pair representation *cannot* represent the empty set ($r \in \gamma(r, m)$), thus there is no meaningful meet (as intersection of incompatible classes is empty, which cannot be represented without overapproximation). Thus, the congruence domain is only join-semilattice, *without* a $\perp$. It already has EXACTORDER  and a *strong* α -relation ; the join is $(r_1, m_1) \sqcup (r_2, m_2) \triangleq (r_1, \gcd(m_1, m_2, r_2 - r_1))$ and JOINISLUB holds . For partial operations whose concrete result can genuinely be empty (division), we wrap the codomain in **WithBottom**(cong) (§4.2, dual of **WithTop**); **WithBottom** preserves EXACTORDER and the strong α -relation. Note that **WithBottom** is not needed for the reduced product between congruence and intervals, as intervals have natural representatives for bottom.

Verified transfer functions. **add** , **opp** , **sub**  are *exact* . Granger's classical formula $(r_1, m_1) + (r_2, m_2) = (r_1 + r_2, \gcd(m_1, m_2))$ clearly overapproximates the sum; exactness requires the converse (i.e., that every element of the result decomposes as a sum of one element from each operand), which follows from Bézout's identity. Negation is a bijection on γ , so it is exact pointwise, and subtraction is exact by composing the two. **mul**  is best but not exact : Granger's formula $(r_1 r_2, \gcd(r_1 m_2, r_2 m_1, m_1 m_2))$ is the tightest abstract result, yet $\gamma(1, 6) \cdot \gamma(1, 10) \subseteq \gamma(1, 2)$ contains 3, while no concrete pair (c_1, c_2) in those classes multiplies to 3. Floor division **div**  and truncating quotient **quot**  (Fig. 2) are best but not exact , with several exact subcases. Two cases (an empty quotient set, and a divisibility-friendly constant divisor) are exact; the remaining three are best-only, defaulting to $\top = (0, 1)$, which we can show is

$$\begin{aligned}
& \text{cong_quot}(r_1, m_1)(r_2, m_2) \triangleq \\
& \begin{cases} \perp & \text{if } m_2 = 0 \wedge r_2 = 0 \quad [\text{exact}] \\ (r_1 \text{ quot } r_2, m_1 \text{ quot } r_2) & \text{if } m_2 = 0 \wedge r_2 \neq 0 \wedge (m_1 = 0 \vee (r_2 \mid m_1 \wedge r_2 \mid r_1)) \quad [\text{exact}] \\ (0, 1) & \text{if } m_2 = 0 \wedge r_2 \neq 0, \text{ otherwise} \quad [\text{best}] \\ (0, \text{quot_gcd_compute}(r_1, r_2, m_2)) & \text{if } m_2 \neq 0 \wedge m_1 = 0 \quad [\text{best}] \\ (0, 1) & \text{if } m_2 \neq 0 \wedge m_1 \neq 0 \quad [\text{best}] \end{cases} \\
& \text{quot_gcd_compute}(r_1, r_2, m_2) \triangleq, \text{ with } r_m \triangleq r_2 \bmod |m_2| \text{ and } g(d) \triangleq \\
& \text{quot_gcd_progression}(|r_1|, d, |m_2|): \\
& \text{quot_gcd_compute} = \begin{cases} g(|m_2|) & \text{if } r_m = 0 \\ \text{gcd}(g(r_m), g(|m_2| - r_m)) & \text{otherwise} \end{cases} \\
& \text{quot_gcd_progression}(n, d, \text{step}) = \begin{cases} 0 & \text{if } n < d \\ n/d & \text{if } d \leq n < d + \text{step} \\ 1 & \text{if } d + \text{step} \leq n \end{cases}
\end{aligned}$$

Fig. 2. Best abstraction of truncating division.

best because two consecutive integer quotients differ by at most 1, except when the dividend is a constant.

The methodologically interesting case is the constant-by-non-constant one, $m_1 = 0 \wedge m_2 \neq 0$. The quotient set is $Q \triangleq \{r_1 \text{ quot } c \mid c \in \gamma(r_2, m_2), c \neq 0\}$ which contains 0 for every c with $|c| > |r_1|$, so the best abstract residue is 0 and the best abstract modulus is $\text{gcd } Q = \text{gcd}\{ \lfloor |r_1|/|c| \rfloor \mid c \in \gamma(r_2, m_2), 1 \leq |c| \leq |r_1| \}$. Q is finite, so a fuel-bounded iteration enumerating its terms is enough to compute $\text{gcd } Q$ — and this is what the original verified implementation did. We make it constant-time by observing that, in any arithmetic progression $d, d + \text{step}, \dots$, as soon as two terms fall in $[1, n]$ (i.e. $d + \text{step} \leq n$), two consecutive quotients $\lfloor n/c \rfloor$ differ by 1 and their gcd contribution collapses to 1; otherwise there is at most one term, contributing 0 when $n < d$, or $\lfloor n/d \rfloor$ when $d \leq n < d + \text{step}$. The transformation to this constant-time computation was the main human-driven algorithmic insight when developing Congruence (§5.6).

5.4 $\mathbb{Z}$ intervals: best by composition

The interval domain is the running example of §4, where its full construction is developed: `itv` is assembled as `Conjunction` of two `WithTop`-extended $\mathbb{Z}$ -bound domains, and its Galois-insertion shapings `nbitv` (for total operations) and `itv_canon` (for partial ones) are obtained via `NonEmpty` and `CanonicalBottom` respectively. The structural lever for the transfer-function proofs is that each bound carries a *weak* α -relation, that can prove `BESTABSTRACTION` only if a set is bounded or when we can find arbitrarily large elements; and `BESTABSTRAC-`

tion composes through **Conjunction** (Lemma 6), so proving an interval is best decomposes into proving each bound is best.

Verified transfer functions. On **nbitv**, negation , addition , and subtraction  are all *exact*: negation is a γ -bijection (swap and negate the bounds), addition is the pointwise sum on bounds, and subtraction is obtained by composition of addition with negation. Comparisons $\leq$  and $=$  are verified directly as exact $\mathbb{Z} \rightarrow \text{bool}$ transfer functions feeding into **qv**; the remaining relations $<, \neq, \geq, >$ are obtained by composition with the (exact) boolean **not** on **qv** and need no separate verification. Multiplication and truncating quotient **Z.quot** are best but not exact. Both proofs proceed by sign-classification of each operand (positive, negative, across-zero) and rely on interval decomposition + assembly using Lemma 2, turning the union of two best abstractions into the best abstraction of the union. Interestingly, we used the proof to optimize the best abstraction when the divisor interval is across-zero: in that case, we know that 1 and -1 will be the “best” divisors, and we can return the best interval without doing any division (for instance, **interval_quot** $[l_2, h_2] [l_1, h_1]$ when $l_1 < 0 < h_1$ and $0 \leq l_2 \leq h_2$ is $[-h_2, h_2]$).

5.5 The reduced product of intervals and congruences

The reduced product **itv** $\wedge$ **cong** combines both domains. The unreduced carrier is the **Conjunction** of the interval and congruence domains ; the shaping combinators of §4 then carve out the user-facing domains, and we recover EXACTORDER (hence a Galois insertion) via **MaximallyReducedSubset** (§4.4): only those pairs $([l, h], (r, m))$ are kept for which neither component can be sharpened. The maximal reduction **reduce**  snaps each interval bound to the congruence grid; we prove that it is *optimal*: for every product element p , **reduce**(p) is a **BESTABSTRACTION** of $\gamma(p)$  — the relational form of the classical recipe $\alpha \circ \gamma$. This is the concrete instance that motivated the **REDUCTION** definition of Section 2.2. Combined with sound componentwise transfer functions, reduction already yields results more precise than running the two domains separately [2]; the rest of this section is about doing better, and about when the reduction step can be skipped.

Addition is best, compositionally. Soundness in the reduced product is just a stepping stone: the interesting question is whether the componentwise transfer functions are *best*. A direct proof of **BESTABSTRACTION** on the product looks unpleasant: each component’s optimality is stated against its own concretization, while the product needs optimality against the *joint* concretization $\gamma(i) \cap \gamma(c)$. This is where α -completeness (§3.2) earns its keep: because it quantifies over arbitrary input sets, it splits componentwise through **Conjunction** (Lemma 6). From α -completeness of interval addition  and of congruence addition , we obtain directly:

Lemma 9 (Product addition is α -complete ). *Let S_2, S_1 be non-empty sets of integers. If the pairs (i_2, c_2) and (i_1, c_1) are best abstractions of S_2 and S_1 in*

the product, then $(\text{interval_add } i_2 \ i_1, \text{cong_add } c_2 \ c_1)$ is a best abstraction of $\{a + b \mid a \in S_2, b \in S_1\}$.

Three corollaries follow. First, on the reduced domain (where every element is maximally reduced), addition is *best* . Second, the componentwise result is itself already maximally reduced : *no reduction step is needed after an addition*, just as none is needed after a join (Lemma 2); once again the optimality proof removes work from the implementation (C3, §5.7). Third, the argument is fully compositional: at no point did we reason about how interval bounds interact with congruence grids.

Exactness does not survive the product. Both component additions are exact, yet the best addition on the product is *not* exact . Take $\{0, 8\} + \{0, 3\}$: each operand is exactly representable ($[0, 8] \cap 8\mathbb{Z}$ and $[0, 3] \cap 3\mathbb{Z}$), but the collecting sum $\{0, 3, 8, 11\}$ is not, as any product element enclosing it must have an interval covering $[0, 11]$ and a modulus dividing $\gcd(3, 8) = 1$: the enclosure also contains 1. Exactness is thus a per-domain property that combination can destroy (C2, §5.7).

The across-divisor division illustrates the kind of interaction worth modelling. In *itv* alone, dividing a constant c by a divisor whose interval crosses zero is best abstracted by $(-|c|, |c|)$ without performing any division, since divisors ± 1 witness the magnitude. In the reduced product, however, the divisor lies in $[l_2, h_2] \cap (r_2 + m_2 \mathbb{Z})$, and the congruence may rule out ± 1 : if the divisor must be even, the smallest possible $|d|$ is 2, so the best result is $(-|c|/2, |c|/2)$, not $(-|c|, |c|)$. The optimisation that was closed-form in the standalone interval domain becomes a genuine arithmetic computation depending on both the interval bound and the congruence modulus.

5.6 Proof methodology

Three levels of proof. Proofs operate at three levels: the *abstract* level (orders and α -relations), the *set-algebraic* level (set operations on γ images), and the *element-wise* level (quantification over concrete elements). The strategy is to stay abstract when possible, since the framework’s compositional lemmas (Lemma 6, ISALPHA on *WithTop*, JOINISLUB-driven joins) discharge obligations without case analysis. Two tactics implement the descent: *to_set* rewrites abstract notations into set-algebraic statements, and *unfold_set* unfolds set membership into element-wise quantifications using typeclasses [37, 38], similarly to *set_unfold* in *std++*. We work without propositional or set extensionality, so $\sqsubseteq$ (rather than equality) is the working notion of set equality; the cost is real but localised at the set-algebraic level and does not propagate to abstract-level proofs.

*Why *ssreflect* [39] and reflection.* The development is heavily algebraic and case-heavy (sign quadrants, divisor/dividend classifications, per-bit splits). *ssreflect*’s small-scale rewriting and disciplined case analysis suit both axes: rewriting chains through $\sqsubseteq$, $\sqsubset$, $\sqsubseteq$ and the compositional lemmas, while *case* discharges wide

classifications without the proof-script explosion of nested `destruct`. Reflection is complementary: on finite domains (notably quadrivalent), our `AutoReflect` type class reflects a `Prop` goal P to a boolean witness b whose value is computed by the kernel, so proofs reduce to `reflexivity`.

LLM assistance for proof and transfer-function synthesis. The initial development (§3, §4, the quadrivalent abstraction, and the exact interval transfer functions) was carried out with almost no LLM assistance, save for occasional questions to ChatGPT or Gemini about Rocq idioms. When proving the harder transfer functions, our experience matched in shape what Jourdan et al. [15] report for Verasco: the bulk of the line count is in large case analyses and arithmetic side conditions. We experimented with LLMs (CLAUDE OPUS 4.5 to 4.8 as they were released, with DEEPSEEK-V4-PRO as a fallback, driven through the CLAUDE CODE tool), and found that generated proof completion was quite effective for these proofs. Once the lemma statement and the leading case-analysis skeleton are in place, LLM-driven completion often discharges the residual `lia/nia/ring` obligations and locates the right `Z`-arithmetic lemma; the 16-way per-bit splits in `KnownBits.v`, the sign-quadrant reductions in interval `quot`, and most congruence operations were filled this way. Our trusted base, namely, concretization definitions, concrete operators like `xorb` or `Z.add`, and the soundness and precision specifications (§3), was written manually.

LLM-generated proofs are usable but often inelegant: the model defaults to brute-forcing element-wise reasoning even when a one-line abstract-level step would close the goal, fails to identify reusable concepts, and re-proves the same intermediate steps several times. On large theorems (multiplication, division), the context grows beyond one session, so splitting into independent sub-cases was essential, and review and minimisation were the real human cost.

The LLM can also synthesise transfer functions, not just their proofs; in well-documented domains like congruence [19], the synthesised functions were often good. It does, however, miss key optimisations: the constant-time `quot_gcd_compute` of Fig. 2 (replacing an earlier fuel-bounded recursion) and the across-zero division of §5.4 were both human-driven algorithmic insights. We see this interaction as comparable to a synthesis loop, but *guided* and *grounded in established proof obligations* rather than example-based SMT search [6, 8].

5.7 Conclusions

We draw three conclusions from this case study.

- C1. Axiom-free verification of precision is possible in plain Rocq.** Our framework based on α -relations can verify `BESTABSTRACTION` and `EXACTLYREPRESENTS` on the transfer functions we set out to test, on unbounded integer domains whose proofs would resist finite SMT or bit-blasting [6, 8], with no axiom and no dependency outside Rocq’s standard library.
- C2. Exactness is a useful precision target.** Many transfer functions in our (exactness-friendly) sample are *exact*, not merely best; when they are not,

many subcases are. Exactness is useful as a proof method, as it amounts to finding elements in the pre-image. It is also useful to prove absence of exactness, as this justifies the need to concentrate on proving “best” using an α -relation. In theory exact may not imply best; but when a constructed domain fails EXACTORDER, the framework’s recipes (`NonEmpty`, `CollapsedBottom`, `CanonicalBottom`, `MaximallyReducedSubset`) recover it, so the precision hierarchy operates well in practice.

C3. Optimality proofs guide implementation. Every transfer function we verify runs in constant time and each case requires only a small number of arithmetic operations. Cases that would probably be overlooked in a sound-only framework, like division of a constant by a non-constant congruence, are properly handled. Structural invariants (`non_bottom`, `MAXIMALLYREDUCED`) let the implementation skip work that a generic recipe would perform: no reduce-after-join, no reduce-after-add on the reduced product (§5.5), no syntactic emptiness test; two non-trivial optimizations regarding interval division were validated by the soundness and optimality proofs.

In addition, we observe (as of early 2026) that LLMs are effective at deriving important parts of our Rocq proofs, but still require human guidance to make the generated code more efficient and the proofs shorter and more elegant.

6 Related work

Verifying transfer functions in interactive proof assistants Monniaux [9] is the first attempt at mechanized verification of an abstract interpretation, where he follows the Galois-connection framework. He notes that this theoretical framework requires adding classical axioms; the development also uses extensionality axioms to simplify proofs. At the time, this prevented the extraction of the Rocq development to OCaml. The development remains small (Rocq has evolved a lot in almost 30 years): only the sign abstract domain is fully verified, with intervals partially verified.

Pichardie [12] studied this problem in depth, and his PhD thesis explains in detail the constructibility issues of the α function (Rocq’s type system makes sure that computable objects cannot depend on logical, uncomputable objects, like a set described by a predicate). He also explains how reasoning with α functions, even outside of the constructibility issues, is fundamentally more difficult than reasoning using γ only: γ -based proofs only require checking that a property holds, while α -based proofs require reasoning about optimality. For these reasons, he proposes to formally verify abstract interpreters using a γ -only framework [11] and to verify only soundness of the analysis, so as to simplify the formal verification.

This style of abstract interpretation has since been the standard in mechanized proofs of abstract interpretation (e.g. [14, 16, 40]). The largest such effort is the Verasco static analyzer [15], a mechanized abstract interpreter for C able to formally verify the absence of undefined behaviour in C programs (based on the

C semantics of COMPCERT [41]). They note that out of the 34k Coq development, “the bulk of the development is the abstract domains for states and (especially) for numbers, which involve large case analyses and difficult proofs over integer and F.P. arithmetic”. This is also our observation; we additionally observe that, in our development, the exactness and best-abstraction proofs are noticeably longer than the corresponding soundness proofs.

There has been some work on formal verification of static analysis properties beyond soundness. Cachera and Pichardie [13] mention that there are three causes of failures in a static analyser: soundness (the most critical), non-termination, and imprecision, and address the first two. But the only mechanized verification effort that specifically targets precision and avoids the use of classical axioms is Darais and Horn [10]. To avoid the constructibility issues of α (a function from a set of concrete elements to an abstract element), they replace α by an extraction function η from a single concrete element to an abstract one. Based on this framework, they prove optimality of transfer functions in the calculational style of Cousot [42]. However, as they acknowledge, **not all classical Galois connections can be recovered by constructive ones**. In their framework, $\eta : A \rightarrow B$ acts on individual concrete elements, and the recovered classical α is its pointwise lifting $\alpha(S) = \{\eta(c) \mid c \in S\}$ —a **set** of η -images, with no aggregation step. Classical numerical domains like intervals and congruences fall outside this shape: the classical interval $\alpha(S) = [\inf S, \sup S]$ produces a single abstract element via aggregation (min/max), and the classical congruence α likewise aggregates via gcd of pairwise differences. Even granting a pointwise embedding $\eta(c) = [c, c]$ for intervals (or $\eta(c) = (c, 0)$ for congruences), the recovered $\alpha(S) = \{[c, c] \mid c \in S\}$ is the set of singleton intervals, not the bounding interval $[\inf S, \sup S]$. The classical Galois connection of these domains is therefore not recoverable in the constructive framework.

To summarize, “due to foundational issues, the abstract interpretation framework is not amenable to mechanized verification while also supporting program extraction using constructive logic” [10]. *Our insight is that a classical proof requiring a Galois connection can be replaced by a constructive proof that needs no extra axiom, by encoding Galois connections via α -relations rather than total α -functions, and by proving the strongest precision property each domain admits—often exactness.* The α -relations are still proper morphisms with respect to set equivalence (i.e., *Proper* ($\sqsubseteq \Rightarrow$ iff)), so the rewriting machinery one expects from a Galois connection survives; what is given up is only the ability to represent α as a total function, which transfer-function precision proofs do not actually need. This alternative framework works well, as shown by our verification of the precision of different transfer functions on different abstract domains.

Synthesis and automated verification of transfer functions A complementary line of work attacks transfer-function precision from the opposite direction: rather than proving optimality interactively, it synthesises or checks transfer functions automatically, typically via automated logic solvers. The root of this line of work is *symbolic abstraction*: expressing concretization and abstract operations in a logical fragment that automated solvers (e.g., SMT) can handle. There

are many instances of this common framework, either specialised to a specific abstraction or parameterised by generic requirements (e.g., availability of a decision procedure, or absence of infinite ascending chains). Reps and Thakur [5] provides a comprehensive overview of these methods. In the following, we focus on more recent work, that can in particular target known-bit and (finite) interval domains.

The AMURTH [6] tool uses a synthesis algorithm guided by counter-examples: concrete elements, classified according to whether they belong to the concretization of the result of the transfer function, help generate a novel candidate transfer function using a user-provided sketch and set of primitives. This candidate function is then verified for soundness, and checked under a bounded approximation for maximal precision (in the concrete, i.e., using MOSTPRECISE). An extension of the method [7] makes it scale to reduced products.

NICETOMEETYOU Peng et al. [8] proposes to synthesise a domain transformer as an intersection of transfer functions. The algorithm generates random abstract functions (guided by a custom metric used to quantify imprecision), and whenever it finds a sound one, intersects it with the already-known sound transfer functions. One benefit of this approach, beyond the fact that it is automatic, is that it can be used to improve the precision of existing transformers.

These approaches are powerful but operate under constraints that our framework is designed to lift. First, SMT-driven synthesis and checking are essentially limited to finite domains, like bitvector abstractions, whereas our development handles infinite domains uniformly, including possibly-unbounded integer intervals and arithmetic congruences. Second, our approach, rooted in abstract interpretation theory, allows definitive proofs of best-precision; furthermore we have identified and hierarchised several notions of best-precision (like EXACTLYREPRESENTS and BESTABSTRACTION), which are strictly stronger than MOSTPRECISE, and we have systematically verified the strongest property each domain admits. Third, synthesised code has no reason to be either readable or efficient, while proof obligations drive the human developer toward a maximally precise yet idiomatic and performant implementation, more likely to reach the quality required for integration in mature compilers or static analyzers. Although our approach is not fully automated, recent LLM-based assistants can discharge many of the simpler transfer-function obligations.

7 Conclusion

Mechanized verification of the optimality of transfer functions in a proof assistant based on the Calculus of Inductive Constructions is possible, without requiring any extra axioms. The key is just to replace the α function of the Galois connection by α -relations, and to reason explicitly modulo equivalence relations. We build an algebraic hierarchy based on this idea; recover relational variants of optional key properties (EXACTORDER, corresponding to Galois insertion, and JOINISLUB); and build domain combinators which help preserve or recover these properties. We also define a hierarchy of precision notions, where “exact” is stronger than “best”

when we have EXACTORDER, and both are stronger than “most precise”. We then implement sound transfer functions for different abstract domains (notably, intervals, known bits and congruences), and prove them optimal – which, in many cases, means exact. We have made the transfer functions more efficient with the help of the proof to validate our changes, and used LLMs to synthesize some of the proofs and transfer functions, which helped, although human interaction remained essential.

The core theory for the Rocq development that we have built, ROCQ BOTTOM, is mature enough for all the optimality proofs that we have tackled. A practical benefit of the constructive, axiom-free style is that verified transfer functions are directly *executable*: the computational layer (carrier types, lattice operations, transfer functions) is separated from the mathematical one and extracts to plain, type-safe OCaml — exercised today on the quadrivalent domain, checked against a module-type signature — with extraction to C (CertiCoq [46]) or C++ (Crane) also possible. The next steps are to make the library more complete by implementing more transfer functions (proving them only sound when necessary), and to package the extracted — sound, optimal, fast, and portable — transfer functions for reuse in static analysers and compilers.

We also want to add more abstractions. We plan on developing certified transfer functions for modular arithmetic; this should be able to reuse our integer abstractions without the need to redo the hardest proofs (in particular, we can ignore the infinite bounds in modular arithmetic). We are particularly interested in completing the triple reduced product between intervals, congruences, and known bits: our analysis of addition (§5.5) already shows that its reduction step needs to flow only from intervals and congruences toward known bits, never back — an example of how α -completeness and exactness proofs tell the implementation which computations and reduction steps can be skipped entirely.

Finally, it would be interesting to improve the framework to allow proofs in the calculational design of abstract interpreters [42], but in a relational setting: i.e., deriving the witness $f^\#(a)$ from the specification $\exists a', \mathcal{R}_\alpha(a', f(\gamma(a)))$ by rewriting, as the classical derivation does from $\alpha(f(\gamma(a)))$. This demands at least a good setoid-rewriting infrastructure, since in our axiom-free development the rewriting happens modulo set equivalence $\sqsubseteq$ rather than equality.

Acknowledgements. We thank the anonymous SAS reviewers, whose suggestions substantially improved this paper. With apologies to Robert Wyatt.

Data availability. The ROCQ BOTTOM development will be available as free software, and the artifact evaluated for this paper is archived at [10.5281/zenodo.21270873].

Bibliography

- [1] Jubi Taneja, Zhengyang Liu, and John Regehr. Testing static analyses for precision and soundness. In *CGO '20: 18th ACM/IEEE International Symposium on Code Generation and Optimization, San Diego, CA, USA, February, 2020*, pages 81–93. ACM, 2020. <https://doi.org/10.1145/3368826.3377927>.
- [2] Patrick Cousot and Radhia Cousot. Systematic design of program analysis frameworks. In *6th ACM Symposium on Principles of Programming Languages*, POPL 1979, pages 269–282, 1979. <https://doi.org/10.1145/567752.567778>.
- [3] Susanne Graf and Hassen Saïdi. Construction of abstract state graphs with PVS. In Orna Grumberg, editor, *Computer Aided Verification, 9th International Conference, CAV '97, Haifa, Israel, June 22-25, 1997, Proceedings*, volume 1254 of *Lecture Notes in Computer Science*, pages 72–83. Springer, 1997. https://doi.org/10.1007/3-540-63166-6_10.
- [4] Thomas W. Reps, Shmuel Sagiv, and Greta Yorsh. Symbolic implementation of the best transformer. In Bernhard Steffen and Giorgio Levi, editors, *5th International Conference on Verification, Model Checking, and Abstract Interpretation – VMCAI 2004*, volume 2937 of *Lecture Notes in Computer Science*, pages 252–266. Springer, 2004. https://doi.org/10.1007/978-3-540-24622-0_21.
- [5] Thomas W. Reps and Aditya V. Thakur. Automating abstract interpretation. In Barbara Jobstmann and K. Rustan M. Leino, editors, *Verification, Model Checking, and Abstract Interpretation - 17th International Conference, VMCAI 2016, St. Petersburg, FL, USA, January 17-19, 2016. Proceedings*, *Lecture Notes in Computer Science*, pages 3–40. Springer, 2016. https://doi.org/10.1007/978-3-662-49122-5_1.
- [6] Pankaj Kumar Kalita, Sujit Kumar Muduli, Loris D’Antoni, Thomas W. Reps, and Subhajit Roy. Synthesizing abstract transformers. *Proc. ACM Program. Lang.*, 6(OOPSLA2):1291–1319, 2022. <https://doi.org/10.1145/3563334>.
- [7] Pankaj Kumar Kalita, Thomas W. Reps, and Subhajit Roy. Synthesizing abstract transformers for reduced-product domains. In Roberto Giacobazzi and Alessandra Gorla, editors, *Static Analysis - 31st International Symposium, SAS 2024, Pasadena, CA, USA, October 20-22, 2024, Proceedings*, volume 14995 of *Lecture Notes in Computer Science*, pages 147–172. Springer, 2024. https://doi.org/10.1007/978-3-031-74776-2_6.
- [8] Xuanyu Peng, Dominic Kennedy, Yuyou Fan, Ben Greenman, John Regehr, and Loris D’Antoni. Nice to meet you: Synthesizing practical MLIR abstract transformers. *Proc. ACM Program. Lang.*, 10(POPL):2323–2351, 2026. <https://doi.org/10.1145/3776722>.
- [9] David Monniaux. Réalisation mécanisée d’interpréteurs abstraits. Master’s thesis, ENS, 1998.

- [10] David Darais and David Van Horn. Constructive galois connections: taming the galois connection framework for mechanized metatheory. In Jacques Garrigue, Gabriele Keller, and Eijiro Sumii, editors, *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*, pages 311–324. ACM, 2016. <https://doi.org/10.1145/2951913.2951934>.
- [11] Patrick Cousot and Radhia Cousot. Abstract interpretation frameworks. *Journal of Logic and Computation*, 2(4):511–547, 1992.
- [12] David Pichardie. *Interprétation abstraite en logique intuitionniste: extraction d’analyseurs Java certifiés*. PhD thesis, Université de Rennes I, 2005.
- [13] David Cachera and David Pichardie. A certified denotational abstract interpreter. In Matt Kaufmann and Lawrence C. Paulson, editors, *Interactive Theorem Proving, First International Conference, ITP 2010, Edinburgh, UK, July 11-14, 2010. Proceedings*, Lecture Notes in Computer Science, pages 9–24. Springer, 2010. https://doi.org/10.1007/978-3-642-14052-5_3.
- [14] Sandrine Blazy, Vincent Laporte, André Oliveira Maroneze, and David Pichardie. Formal verification of a C value analysis based on abstract interpretation. *CoRR*, abs/1304.3596, 2013.
- [15] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. A formally-verified c static analyzer. *ACM SIGPLAN Notices*, 50(1):247–259, 2015.
- [16] Sébastien Michelland, Yannick Zakowski, and Laure Gonnord. Abstract interpreters: A monadic approach to modular verification. *Proc. ACM Program. Lang.*, 8(ICFP):602–629, 2024. <https://doi.org/10.1145/3674646>.
- [17] Anders Møller and Michael I. Schwartzbach. Static program analysis, 10 2018. Department of Computer Science, Aarhus University, <http://cs.au.dk/~amoeller/spa/>.
- [18] Antoine Miné. Tutorial on static inference of numeric invariants by abstract interpretation. *Found. Trends Program. Lang.*, 4(3-4):120–372, 2017. <https://doi.org/10.1561/25000000034>.
- [19] Philippe Granger. Static analysis of arithmetical congruences. *International Journal of Computer Mathematics*, 30(3-4):165–190, 1989. <https://doi.org/10.1080/00207168908803778>.
- [20] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Sound, precise, and fast abstract interpretation with tristate numbers. In Jae W. Lee, Sebastian Hack, and Tatiana Shpeisman, editors, *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2022, Seoul, Korea, Republic of, April 2-6, 2022*, pages 254–265. IEEE, 2022. <https://doi.org/10.1109/CGO53902.2022.9741267>.
- [21] François Garillot, Georges Gonthier, Assia Mahboubi, and Laurence Rideau. Packaging mathematical structures. In Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel, editors, *Theorem Proving in Higher Order Logics, 22nd International Conference, TPHOLs 2009, Munich, Germany, August 17-20, 2009. Proceedings*, volume 5674 of *Lecture Notes in Computer Science*, pages 327–342. Springer, 2009. https://doi.org/10.1007/978-3-642-03359-9_23.

- [22] Bas Spitters and Eelis van der Weegen. Type classes for mathematics in type theory. *CoRR*, abs/1102.1323, 2011.
- [23] Philippe Granger. Improving the results of static analyses programs by local decreasing iteration. In R. K. Shyamasundar, editor, *Foundations of Software Technology and Theoretical Computer Science, 12th Conference, New Delhi, India, December 18-20, 1992, Proceedings*, volume 652 of *Lecture Notes in Computer Science*, pages 68–79. Springer, 1992. https://doi.org/10.1007/3-540-56287-7_95.
- [24] Marie Pelleau, Antoine Miné, Charlotte Truchet, and Frédéric Benhamou. A constraint solver based on abstract domains. In Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni, editors, *Verification, Model Checking, and Abstract Interpretation, 14th International Conference, VMCAI 2013, Rome, Italy, January 20-22, 2013. Proceedings*, Lecture Notes in Computer Science, pages 434–454. Springer, 2013. https://doi.org/10.1007/978-3-642-35873-9_26.
- [25] Dorian Lesbre and Matthieu Lemerre. Compiling with abstract interpretation. *Proc. ACM Program. Lang.*, 8(PLDI):368–393, 2024. <https://doi.org/10.1145/3656392>.
- [26] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. Equality saturation: A new approach to optimization. *Log. Methods Comput. Sci.*, 7(1), 2011. [https://doi.org/10.2168/LMCS-7\(1:10\)2011](https://doi.org/10.2168/LMCS-7(1:10)2011).
- [27] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.*, 5(POPL):1–29, 2021. <https://doi.org/10.1145/3434304>.
- [28] Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Conference Record of the Fifth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 84–97, Tucson, Arizona, 1978. ACM Press, New York, NY. <https://doi.org/10.1145/512760.512770>.
- [29] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2 edition, 2002.
- [30] Roberto Giacobazzi and Elisa Quintarelli. Incompleteness, counterexamples, and refinements in abstract model-checking. In Patrick Cousot, editor, *Static Analysis, 8th International Symposium, SAS 2001, Paris, France, July 16-18, 2001, Proceedings*, volume 2126 of *Lecture Notes in Computer Science*, pages 356–373. Springer, 2001. https://doi.org/10.1007/3-540-47764-0_20.
- [31] Antoine Miné. Abstract domains for bit-level machine integer and floating-point operations. In *WING'12 - 4th International Workshop on Invariant Generation*, page 16, Manchester, United Kingdom, 6 2012.
- [32] John Regehr, Alastair Reid, and Kirk Webb. Eliminating stack overflow by abstract interpretation. *ACM Trans. Embed. Comput. Syst.*, 4(4):751–778, 2005. <https://doi.org/10.1145/1113830.1113833>.
- [33] David Monniaux. Verification of device drivers and intelligent controllers: a case study. In Christoph M. Kirsch and Reinhard Wilhelm, editors, *Proceedings of the 7th ACM & IEEE International conference on Embedded*

- software, *EMSOFT 2007, September 30 - October 3, 2007, Salzburg, Austria*, pages 30–36. ACM, 2007. <https://doi.org/10.1145/1289927.1289937>.
- [34] Laurent D. Michel and Pascal Van Hentenryck. Constraint satisfaction over bit-vectors. In Michela Milano, editor, *Principles and Practice of Constraint Programming - 18th International Conference, CP 2012, Québec City, QC, Canada, October 8-12, 2012. Proceedings*, volume 7514 of *Lecture Notes in Computer Science*, pages 527–543. Springer, 2012. https://doi.org/10.1007/978-3-642-33558-7_39.
- [35] Zakaria Chihani, Bruno Marre, François Bobot, and Sébastien Bardin. Sharpening constraint programming approaches for bit-vector theory. In Domenico Salvagnin and Michele Lombardi, editors, *Integration of AI and OR Techniques in Constraint Programming - 14th International Conference, CPAIOR 2017, Padua, Italy, June 5-8, 2017, Proceedings*, volume 10335 of *Lecture Notes in Computer Science*, pages 3–20. Springer, 2017. https://doi.org/10.1007/978-3-319-59776-8_1.
- [36] Jan Onderka and Stefan Ratschan. Fast three-valued abstract bit-vector arithmetic. In Bernd Finkbeiner and Thomas Wies, editors, *Verification, Model Checking, and Abstract Interpretation - 23rd International Conference, VMCAI 2022, Philadelphia, PA, USA, January 16-18, 2022, Proceedings*, *Lecture Notes in Computer Science*, pages 242–262. Springer, 2022. https://doi.org/10.1007/978-3-030-94583-1_12.
- [37] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In Giuseppe Castagna and Andrew D. Gordon, editors, *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, pages 205–217. ACM, 2017. <https://doi.org/10.1145/3009837.3009855>.
- [38] The std++ team. Rocq-std++: An extended standard library for Rocq. <https://gitlab.mpi-sws.org/iris/stdpp>, 2017–2026. Accessed: 2026-05-08.
- [39] Georges Gonthier and Assia Mahboubi. An introduction to small scale reflection in coq. *Journal of Formalized Reasoning*, 3(2):95–152, Jan. 2010. <https://doi.org/10.6092/issn.1972-5787/1979>. URL <https://jfr.unibo.it/article/view/1979>.
- [40] Xavier Leroy. A formally verified compiler back-end. *J. Autom. Reason.*, 43(4):363–446, 2009. <https://doi.org/10.1007/S10817-009-9155-4>.
- [41] Xavier Leroy. Formal certification of a compiler back-end or: programming a compiler with a proof assistant. In J. Gregory Morrisett and Simon L. Peyton Jones, editors, *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2006, Charleston, South Carolina, USA, January 11-13, 2006*, pages 42–54. ACM, 2006. <https://doi.org/10.1145/1111037.1111042>.
- [42] Patrick Cousot. The calculational design of a generic abstract interpreter. In M. Broy and R. Steinbrüggen, editors, *Calculational System Design*. NATO ASI Series F. IOS Press, Amsterdam, 1999.

- [43] Matan Shachnai, Harishankar Vishwanathan, Srinivas Narayana, and Santosh Nagarakatte. Comparing the precision of abstract operators in the eBPF verifier using differential synthesis. In *Static Analysis - 32nd International Symposium, SAS 2025*. Springer, 2025.
- [44] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Automatic synthesis of abstract operators for eBPF. In *Proceedings of the Workshop on eBPF and Kernel Extensions (eBPF '25)*. ACM, 2025.
- [45] Marco Campion, Mila Dalla Preda, Roberto Giacobazzi, and Caterina Urban. A logic for the imprecision of abstract interpretations. *Proceedings of the ACM on Programming Languages*, 10(POPL), 2026. <https://doi.org/10.1145/3776707>.
- [46] Abhishek Anand, Andrew W. Appel, Greg Morrisett, Zoe Paraskevopoulou, Randy Pollack, Olivier Savary Bélanger, Matthieu Sozeau, and Matthew Z. Weaver. Certicoq : A verified compiler for coq. 2016. URL <https://api.semanticscholar.org/CorpusID:9607775>.